

Table of Contents

1 Home

2 Practical sessions

[2.1 Connecting to WIS2 over MQTT](#)

[2.2 Accessing your student VM](#)

[2.3 Initializing wis2box](#)

[2.4 Configuring datasets in wis2box](#)

[2.5 Configuring station metadata](#)

[2.6 Ingesting data for publication](#)

[2.7 Data conversion tools](#)

[2.8 CSV-to-BUFR mapping templates](#)

[2.9 Adding GTS-headers to WIS2 notifications](#)

[2.10 Setting up a recommended dataset with access control](#)

[2.11 Downloading data from WIS2](#)

[2.12 Discovering datasets from the WIS2 Global Discovery Catalogue](#)

3 Cheatsheets

[3.1 Linux](#)

[3.2 Docker](#)

[3.3 WIS2 in a box](#)

4 csv2bufr templates

[4.1 AWS hourly](#)

[4.2 DAYCLI](#)



1 WIS2 in a box training

WIS2 in a box ([wis2box](#)) is a Free and Open Source (FOSS) Reference Implementation of a WMO WIS2 Node. The project provides a plug and play toolset to ingest, process, and publish weather/climate/water data using standards-based approaches in alignment with the WIS2 principles. [wis2box](#) also provides access to all data in the WIS2 network. [wis2box](#) is designed to have a low barrier to entry for data providers, providing enabling infrastructure and services for data discovery, access, and visualization.

This training provides step-by-step explanations of various aspects of the [wis2box](#) project as well as a number of exercises to help you publish and download data from WIS2. The training is provided in the form of overview presentations as well as hands-on practical exercises.

Participants will be able to work with sample test data and metadata, as well as integrate their own data and metadata.

This training covers a wide range of topics (install/setup/configuration, publishing/downloading data, etc.).

1.1 Goals and learning outcomes

The goals of this training are to become familiar with the following:

- WIS2 architecture core concepts and components
- data and metadata formats used in WIS2 for discovery and access
- [wis2box](#) architecture and environment
- [wis2box](#) core functions:
 - metadata management
 - data ingest and transformation to BUFR format
 - MQTT broker for WIS2 message publishing
 - HTTP endpoint for data download
 - API endpoint for programmatic access to data

1.2 Navigation

The left hand navigation provides a table of contents for the entire training.

The right hand navigation provides a table of contents for a specific page.

1.3 Prerequisites

1.3.1 Knowledge

- Basic Linux commands (see the [cheatsheet](#))
- Basic knowledge of networking and Internet protocols

1.3.2 Software

This training requires the following tools:

- An instance running Ubuntu OS (provided by WMO trainers during local training sessions) see [Accessing your student VM](#)
- SSH client to access your instance
- MQTT Explorer on your local machine
- SCP and SFTP client to copy files from your local machine

1.4 Conventions

Question

A section marked like this invites you to answer a question.

Also you will notice tips and notes sections within the text:

Tip

Tips share help on how to best achieve tasks.

Note

Notes provide additional information on the topic covered by the practical session, as well as how to best achieve tasks.

Examples are indicated as follows:

Configuration

```
1 my-collection-defined-in-yaml:  
2   type: collection  
3   title: my title defined as a yaml attribute named title  
4   description: my description as a yaml attribute named description
```

Snippets which need to be typed in a on a terminal/console are indicated as:

```
echo 'Hello world'
```

Container names (running images) are denoted in **bold**.

1.5 Training location and materials

The training contents, wiki and issue tracker are managed on GitHub at <https://github.com/World-Meteorological-Organization/wis2box-training>.

1.6 Printing the material

This training can be exported to PDF. To save or print this training material, go to the [print page](#), and select File > Print > Save as PDF.

1.7 Exercise materials

Exercise materials can be downloaded from the [exercise-materials.zip](#) zipfile.

1.8 Support

For issues/bugs/suggestions or improvements/contributions to this training, please use the [GitHub issue tracker](#).

All wis2box bugs, enhancements and issues can be reported on [GitHub](#).

For additional support of questions, please contact wis2-support@wmo.int.

As always, wis2box core documentation can always be found at <https://docs.wis2box.wis.wmo.int>.

Contributions are always encouraged and welcome!

2 Practical sessions

2.1 Connecting to WIS2 over MQTT



Learning outcomes

By the end of this practical session, you will be able to:

- connect to the WIS2 Global Broker using MQTT Explorer
- review the WIS2 topic structure
- review the WIS2 notification message structure

2.1.1 Introduction

WIS2 uses the MQTT protocol to advertise the availability of weather/climate/water data. The WIS2 Global Broker subscribes to all WIS2 Nodes in the network and republishes the messages it receives. The Global Cache subscribes to the Global Broker, downloads the data in the message and then republishes the message on the `cache` topic with a new URL. The Global Discovery Catalogue publishes discovery metadata from the Broker and provides a search API.

This is an example of the WIS2 notification message structure for a message received on the topic `origin/a/wis2/br-inmet/data/core/weather/surface-based-observations/synop`:

```

{
  "id": "59f9b013-c4b3-410a-a52d-fff18f3f1b47",
  "type": "Feature",
  "version": "v04",
  "geometry": {
    "coordinates": [
      -38.69389,
      -17.96472,
      60
    ],
    "type": "Point"
  },
  "properties": {
    "data_id": "br-inmet/data/core/weather/surface-based-observations/synop/WIGOS_0-76-2-2900801000W83499_20240815T060000",
    "datetime": "2024-08-15T06:00:00Z",
    "pubtime": "2024-08-15T09:52:02Z",
    "integrity": {
      "method": "sha512",
      "value":
"TBuWycx/G0lIiTo47eFPBViGutxcIyk7eikppAKPc4aHgOmTIS5Wb9+0v3awMOyCgwpFhTruRRCVReMQMp5kYw=="
    },
    "content": {
      "encoding": "base64",
      "value":
"QlVGUGAA+gQAABYAACsAAAAAAIAHAAH6AgPBgAAAAALAAABgMGWx1AAAM0ABOIAAA0DM00TkAAAAAAAAAAAAAKb5oKEpJ6YkKJ6mAAAAAAAAAAAAAAAv0QeYA29WQa87ZhH4CQP//z+P//BD////+ASznXuUb///8MgAS3/////8X///e+AP////AB/+R/yf/////////////////6/1/79H/3///gEt////////4BLP6QAf/+pAB//4H0YJ/YeAh/f2///7TH/////9+j//f////////v0f////////wNzc3Nw=="
    },
    "size": 250
  },
  "wigos_station_identifier": "0-76-2-2900801000W83499"
},
{
  "links": [
    {
      "rel": "canonical",
      "type": "application/bufr",
      "href": "http://wis2bra.inmet.gov.br/data/2024-08-15/wis/br-inmet/data/core/weather/surface-based-observations/synop/WIGOS_0-76-2-2900801000W83499_20240815T060000.bufr4",
      "length": 250
    }
  ]
}

```

In this practical session you will learn how to use the MQTT Explorer tool to setup an MQTT client connection to a WIS2 Global Broker and be able to display WIS2 notification messages.

MQTT Explorer is a useful tool to browse and review the topic structure for a given MQTT broker to review data being published.

Note that MQTT is primarily used for "machine-to-machine" communication; meaning that there would normally be a client automatically parsing the messages as they are received. To work with MQTT programmatically (for example, in Python), you can use MQTT client libraries such as [paho-mqtt](#) to connect to an MQTT broker and process incoming messages. There exist numerous MQTT client and server software, depending on your requirements and technical environment.

2.1.2 Using MQTT Explorer to connect to the Global Broker

To view messages published by a WIS2 Global Broker you can "MQTT Explorer" which can be downloaded from the [MQTT Explorer website](#).

Open MQTT Explorer and add a new connection to the Global Broker hosted by MeteoFrance using the following details:

- host: globalbroker.meteo.fr
- port: 8883
- username: everyone
- password: everyone

The screenshot shows the 'MQTT Connection' window in MQTT Explorer. The title bar indicates the connection URL: `mqtt://globalbroker.meteo.fr :8883/`. The interface is divided into several sections:

- Name:** A text field containing 'global-broker-france'.
- Validate certificate:** A toggle switch currently turned off.
- Encryption (tls):** A toggle switch currently turned on.
- Protocol:** A dropdown menu showing 'mqtt://'.
- Host:** A text field containing 'globalbroker.meteo.fr'.
- Port:** A text field containing '8883'.
- Username:** A text field containing 'everyone'.
- Password:** A text field containing 'everyone' with an eye icon for toggling visibility.

At the bottom, there are four buttons: 'DELETE' (with a trash icon), 'ADVANCED' (with a gear icon), 'SAVE' (in a red button), and 'CONNECT' (in a blue button).

Click on the 'ADVANCED' button, remove the pre-configured topics and add the following topics to subscribe to:

- `origin/a/wis2/#`

MQTT Connection

mqtt://globalbroker.meteofrance.fr:8883/

Topic

`origin/a/wis2/#`

QoS

0 ▼

+

ADD

	Topic	QoS
	<code>origin/a/wis2/#</code>	0

Note

When setting up MQTT subscriptions you can use the following wildcards:

- **Single-level (+)**: a single-level wildcard replaces one topic level
- **Multi-level (#)**: a multi-level wildcard replaces multiple topic levels

In this case `origin/a/wis2/#` will subscribe to all topics under the `origin/a/wis2` topic.

Click 'BACK', then 'SAVE' to save your connection and subscription details. Then click 'CONNECT':

Messages should start appearing in your MQTT Explorer session as follows:

☰ MQTT Explorer

🔍 Search...

⏸ DISCONNECT



▼ globalbroker.meteo.fr

▶ origin (26 topics, 208 messages)

Topic
Value

You are now ready to start exploring the WIS2 topics and message structure.

2.1.3 Exercise 1: Review the WIS2 topic structure

Use MQTT to browse topic structure under the `origin` topics.

? Question

How can we distinguish the WIS centre that published the data?

✓ Click to reveal answer

You can click on the left hand side window in MQTT Explorer to expand the topic structure.

We can distinguish the WIS centre that published the data by looking at the fourth level of the topic structure. For example, the following topic:

```
origin/a/wis2/br-inmet/data/core/weather/surface-based-observations/synop
```

tells us that the data was published a WIS centre with the centre-id `br-inmet`, which is the centre-id for Instituto Nacional de Meteorologia - INMET, Brazil.

? Question

How can we distinguish between messages published by WIS-centres hosting a GTS-to-WIS2 gateway and messages published by WIS-centres hosting a WIS2 node?

✓ Click to reveal answer

We can distinguish messages coming from GTS-to-WIS2 gateway by looking at the centre-id in the topic structure. For example, the following topic:

```
origin/a/wis2/de-dwd-gts-to-wis2/data/core/I/S/A/I/01/sbbr
```

tells us that the data was published by the GTS-to-WIS2 gateway hosted by Deutscher Wetterdienst (DWD), Germany. The GTS-to-WIS2 gateway is a special type of data-publisher that publishes data from the Global Telecommunication System (GTS) to WIS2. The topic structure is composed by the TTAAii CCCC headers for the GTS messages.

2.1.4 Exercise 2: Review the WIS2 message structure

Disconnect from MQTT Explorer and update the 'Advanced' sections to change the subscription to the following:

- `origin/a/wis2+/data/core/weather/surface-based-observations/synop`
- `cache/a/wis2+/data/core/weather/surface-based-observations/synop`

MQTT Connection

mqtt://globalbroker.meteo.fr:8883/

Topic

QoS

0 ▼

+ ADD

	Topic	QoS
	origin/a/wis2/+/data/core/weather/surface-based-observations/synop	0
	cache/a/wis2/+/data/core/weather/surface-based-observations/synop	0

 Note

The  wildcard is used to subscribe to all WIS-centres.

Reconnect to the Global Broker and wait for messages to appear.

You can view the content of the WIS2 message in the "Value" section on the right hand side. Try to expand the topic structure to see the different levels of the message until you reach the last level and review message content of one of the messages.

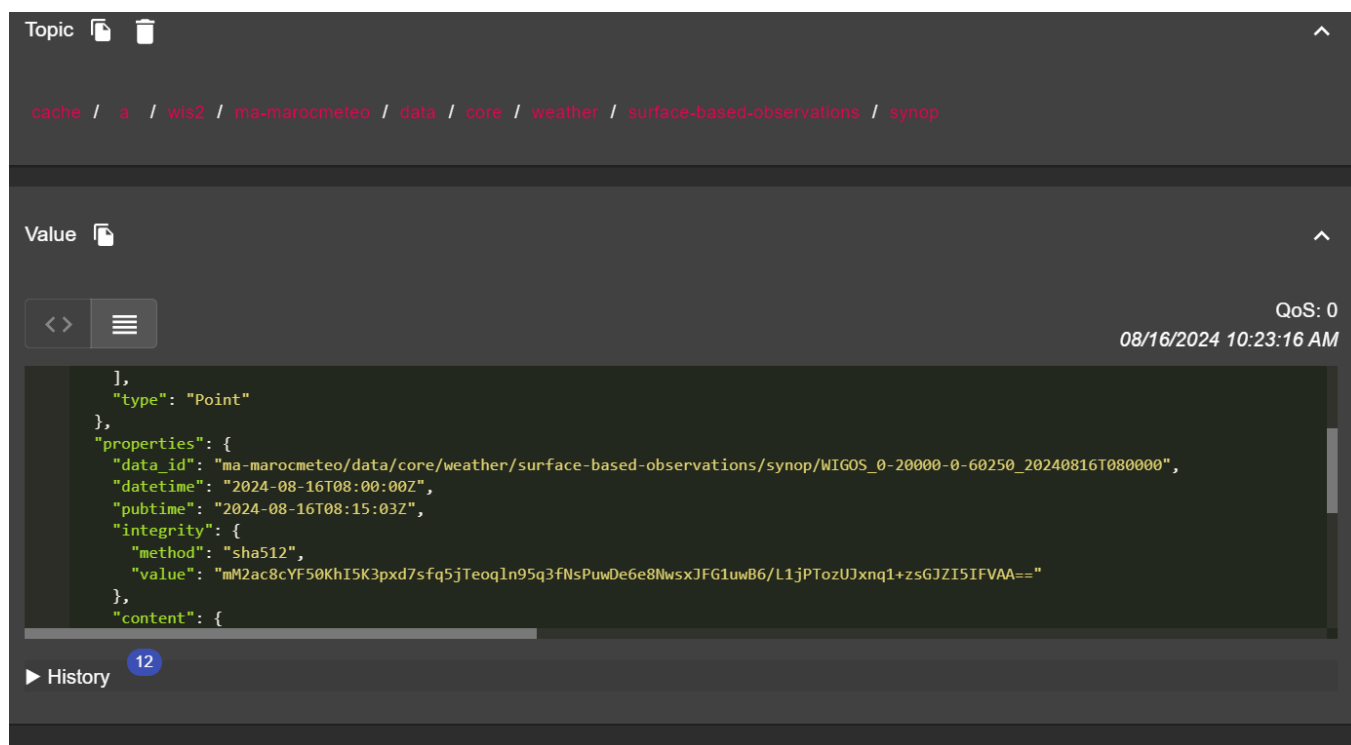
 Question

How can we identify the timestamp that the data was published? And how can we identify the timestamp that the data was collected?

 Click to reveal answer

The timestamp that the data was published is contained in the `properties` section of the message with a key of `pubtime`.

The timestamp that the data was collected is contained in the `properties` section of the message with a key of `datetime`.



The screenshot shows a Kafka message viewer interface. The 'Topic' field displays the path: `cache / a / wis2 / ma-marocmeteo / data / core / weather / surface-based-observations / synop`. The 'Value' field shows a JSON message with the following structure:

```
{
  "type": "Point",
  "properties": {
    "data_id": "ma-marocmeteo/data/core/weather/surface-based-observations/synop/WIGOS_0-20000-0-60250_20240816T080000",
    "datetime": "2024-08-16T08:00:00Z",
    "pubtime": "2024-08-16T08:15:03Z",
    "integrity": {
      "method": "sha512",
      "value": "mM2ac8cYF50KhI5K3pxd7sfq5jTeoqln95q3fNsPuwDe6e8NwsxJFG1uwB6/L1jPTozUJxnq1+zsGJZI5IFVAA=="
    }
  },
  "content": {

```

At the bottom of the viewer, there is a 'History' button and a blue circle containing the number '12'. The top right corner of the viewer shows 'QoS: 0' and the timestamp '08/16/2024 10:23:16 AM'.

 Question

How can we download the data from the URL provided in the message?

 Click to reveal answer

The URL is contained in the `links` section with `rel="canonical"` and defined by the `href` key.

You can copy the URL and paste it into a web browser to download the data.

2.1.5 Exercise 3: Review the difference between 'origin' and 'cache' topics

Make sure you are still connected to the Global Broker using the topic subscriptions

`origin/a/wis2/+data/core/weather/surface-based-observations/synop` and

`cache/a/wis2/+data/core/weather/surface-based-observations/synop` as described in Exercise 2.

Try to identify a message for the same centre-id published on both the `origin` and `cache` topics.

Question

What is the difference between the messages published on the `origin` and `cache` topics?

Click to reveal answer

The messages published on the `origin` topics are the original messages which the Global Broker republishes from the WIS2 Nodes in the network.

The messages published on the `cache` topics are the messages for data has been downloaded by the Global Cache. If you check the content of the message from the topic starting with `cache`, you will see that the 'canonical' link has been updated to a new URL.

There are multiple Global Caches in the WIS2 network, so you will receive one message from each Global Cache that has downloaded the message.

The Global Cache will only download and republish messages that were published on the `../data/core/...` topic hierarchy.

2.1.6 Conclusion

Congratulations!

In this practical session, you learned:

- how to subscribe to WIS2 Global Broker services using MQTT Explorer
- the WIS2 topic structure
- the WIS2 notification message structure
- the difference between core and recommended data
- the topic structure used by the GTS-to-WIS2 gateway
- the difference between Global Broker messages published on the `origin` and `cache` topics

2.2 Accessing your student VM



Learning outcomes

By the end of this practical session, you will be able to:

- access your student VM over SSH and WinSCP
- verify the required software for the practical exercises is installed
- verify you have access to exercise materials for this training on your local student VM

2.2.1 Introduction

As part of locally run wis2box training sessions, you can access your personal student VM on the local training network named "WIS2-training".

Your student VM has the following software pre-installed:

- Ubuntu 22.04.3 LTS [ubuntu-22.04.3-live-server-amd64.iso](#)
- Python 3.10.12
- Docker 24.0.6
- Docker Compose 2.21.0
- Text editors: vim, nano

 Note

If you want to run this training outside of a local training session, you can provide your own instance using any cloud provider, for example:

- GCP (Google Cloud Platform) VM instance `e2-medium`
- AWS (Amazon Web Services) `ec2-instance` `t3a.medium`
- Azure (Microsoft) Azure Virtual Machine `standard_b2s`

Select Ubuntu Server 22.0.4 LTS as OS.

After creating your VM ensure you have installed python, docker and docker compose, as described at [wis2box-software-dependencies](#).

The release archive for wis2box used in this training can be downloaded as follows:

```
wget https://github.com/World-Meteorological-Organization/wis2box-  
release/releases/download/1.0.0/wis2box-setup.zip  
unzip wis2box-setup.zip
```

You can always find the latest 'wis2box-setup' archive at <https://github.com/World-Meteorological-Organization/wis2box/releases>.

The exercise material used in this training can be downloaded as follows:

```
wget https://training.wis2box.wis.wmo.int/exercise-materials.zip  
unzip exercise-materials.zip
```

The following additional Python packages are required to run the exercise materials:

```
pip3 install minio  
pip3 install pywiscat==0.2.2
```

If you are using the student VM provided during local WIS2 training sessions, the required software will already be installed.

2.2.2 Connect to your student VM on the local training network

Connect your PC on the local Wi-Fi broadcasted in the room during WIS2 training as per the instructions provided by the trainer.

Use an SSH client to connect to your student VM using the following:

- **Host:** (provided during in-person training)
- **Port:** 22
- **Username:** (provided during in-person training)
- **Password:** (provided during in-person training)

**Tip**

Contact a trainer if you are unsure about the hostname/username or have issues connecting.

Once connected, please change your password to ensure others cannot access your VM:

```
limper@student-vm:~$ passwd
Changing password for testuser.
Current password:
New password:
Retype new password:
passwd: password updated successfully
```

2.2.3 Verify software versions

To be able to run wis2box, the student VM should have Python, Docker and Docker Compose pre-installed.

Check Python version:

```
python3 --version
```

returns:

```
Python 3.10.12
```

Check docker version:

```
docker --version
```

returns:

```
Docker version 24.0.6, build ed223bc
```

Check Docker Compose version:

```
docker compose version
```

returns:

```
Docker Compose version v2.21.0
```

To ensure your user can run Docker commands your user has been added to the `docker` group.

To test that your user can run docker hello-world, run the following command:


```
docker run hello-world
```

This should pull the hello-world image and run a container that prints a message.

Check that you see the following in the output:

```
...  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
...
```

2.2.4 Inspect the exercise materials

Inspect the contents of your home directory; these are the materials used as part of the training and practical sessions.

```
ls ~/
```

returns:

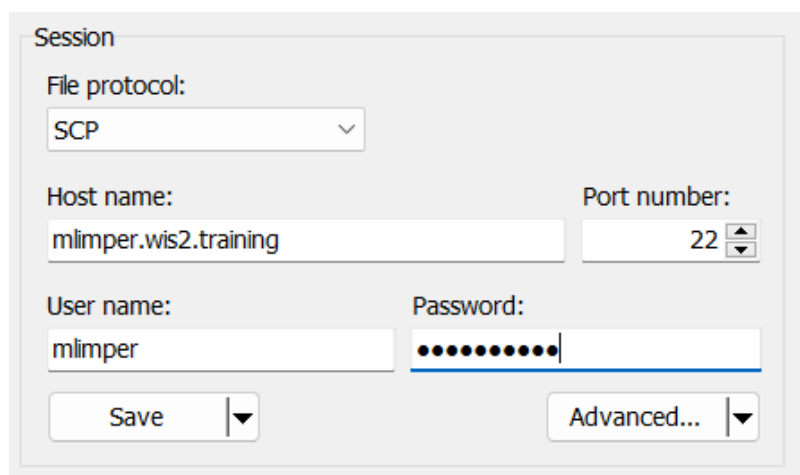
```
exercise-materials  wis2box
```

If you have WinSCP installed on your local PC, you can use it to connect to your student VM and inspect the contents of your home directory and download or upload files between your VM and your local PC.

WinSCP is not required for the training, but it can be useful if you want to edit files on your VM using a text editor on your local PC.

Here is how you can connect to your student VM using WinSCP:

Open WinSCP and click on the "New Site". You can create a new SCP connection to your VM as follows:



The image shows the 'Session' configuration window in WinSCP. It contains the following fields and controls:

- File protocol:** A dropdown menu set to 'SCP'.
- Host name:** A text box containing 'mlimper.wis2.training'.
- Port number:** A spinner box set to '22'.
- User name:** A text box containing 'mlimper'.
- Password:** A text box filled with dots for masking.
- Buttons:** 'Save' and 'Advanced...' buttons at the bottom.

Click 'Save' and then 'Login' to connect to your VM.

And you should be able to see the following content:

/home/mlimper/			
Name	Size	Changed	Rights
		10/5/2023 7:49:55 ...	rwxr-xr-x
 wis2box-1.0b8		10/5/2023 7:50:00 ...	rwxrwxr-x
 exercise-materials		10/4/2023 10:14:28...	rwxr-xr-x

2.2.5 Conclusion

✓ Congratulations!

In this practical session, you learned how to:

- access your student VM over SSH and WinSCP
- verify the required software for the practical exercises is installed
- verify you have access to exercise materials for this training on your local student VM

2.3 Initializing wis2box

Learning outcomes

By the end of this practical session, you will be able to:

- run the `wis2box-create-config.py` script to create the initial configuration
- start wis2box and check the status of its components
- view the contents of the **wis2box-api**
- access the **wis2box-webapp**
- connect to the local **wis2box-broker** using MQTT Explorer

Note

The current training materials are based on wis2box-release 1.0.0.

See [accessing-your-student-vm](#) for instructions on how to download and install the wis2box software stack if you are running this training outside of a local training session.

2.3.1 Preparation

Login to your designated VM with your username and password and ensure you are in the `wis2box` directory:

```
cd ~/wis2box
```

2.3.2 Creating the initial configuration

The initial configuration for the wis2box requires:

- an environment file `wis2box.env` containing the configuration parameters
- a directory on the host-machine to share between the host machine and the wis2box containers defined by the `WIS2BOX_HOST_DATADIR` environment variable

The `wis2box-create-config.py` script can be used to create the initial configuration of your wis2box.

It will ask you a set of question to help setup your configuration.

You will be able to review and update the configuration files after the script has completed.

Run the script as follows:

```
python3 wis2box-create-config.py
```

wis2box-host-data directory

The script will ask you to enter the directory to be used for the `WIS2BOX_HOST_DATADIR` environment variable.

Note that you need to define the full path to this directory.

For example if your username is `username`, the full path to the directory is `/home/username/wis2box-data`:

```
username@student-vm-username:~/wis2box$ python3 wis2box-create-config.py
Please enter the directory to be used for WIS2BOX_HOST_DATADIR:
/home/username/wis2box-data
The directory to be used for WIS2BOX_HOST_DATADIR will be set to:
/home/username/wis2box-data
Is this correct? (y/n/exit)
y
The directory /home/username/wis2box-data has been created.
```

wis2box URL

Next, you will be asked to enter the URL for your wis2box. This is the URL that will be used to access the wis2box web application, API and UI.

Please use `http://<your-hostname-or-ip>` as the URL.

```
Please enter the URL of the wis2box:
For local testing the URL is http://localhost
To enable remote access, the URL should point to the public IP address or domain name of the
server hosting the wis2box.
http://username.wis2.training
The URL of the wis2box will be set to:
http://username.wis2.training
Is this correct? (y/n/exit)
```

WEBAPP, STORAGE and BROKER passwords

You can use the option of random password generation when prompted for and `WIS2BOX_WEBAPP_PASSWORD`, `WIS2BOX_STORAGE_PASSWORD`, `WIS2BOX_BROKER_PASSWORD` and define your own.

Don't worry about remembering these passwords, they will be stored in the `wis2box.env` file in your wis2box-directory.

Review `wis2box.env`

Once the scripts is completed check the contents of the `wis2box.env` file in your current directory:

```
cat ~/wis2box/wis2box.env
```

Or check the content of the file via WinSCP.

Question

What is the value of WISBOX_BASEMAP_URL in the wis2box.env file?

Click to reveal answer

The default value for WIS2BOX_BASEMAP_URL is `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`.

This URL refers to the OpenStreetMap tile server. If you want to use a different map provider, you can change this URL to point to a different tile server.

Question

What is the value of the WIS2BOX_STORAGE_DATA_RETENTION_DAYS environment variable in the wis2box.env file?

Click to reveal answer

The default value for WIS2BOX_STORAGE_DATA_RETENTION_DAYS is 30 days. You can change this value to a different number of days if you wish.

The wis2box-management container runs a cronjob on a daily basis to remove data older than the number of days defined by WIS2BOX_STORAGE_DATA_RETENTION_DAYS from the `wis2box-public` bucket and the API backend:

```
0 0 * * * su wis2box -c "wis2box data clean --days=$WIS2BOX_STORAGE_DATA_RETENTION_DAYS"
```

Note

The `wis2box.env` file contains environment variables defining the configuration of your wis2box. For more information consult the [wis2box-documentation](#).

Do not edit the `wis2box.env` file unless you are sure of the changes you are making. Incorrect changes can cause your wis2box to stop working.

Do not share the contents of your `wis2box.env` file with anyone, as it contains sensitive information such as passwords.

2.3.3 Start wis2box

Ensure you are in the directory containing the wis2box software stack definition files:

```
cd ~/wis2box
```

Start wis2box with the following command:

```
python3 wis2box-ctl.py start
```

When running this command for the first time, you will see the following output:

```
No docker-compose.images-*.yaml files found, creating one
Current version=Undefined, latest version=1.0.0
Would you like to update ? (y/n/exit)
```

Select `y` and the script will create the file `docker-compose.images-1.0.0.yaml`, download the required Docker images and start the services.

Downloading the images may take some time depending on your internet connection speed. This step is only required the first time you start wis2box.

Inspect the status with the following command:

```
python3 wis2box-ctl.py status
```

Repeat this command until all services are up and running.



wis2box and Docker

wis2box runs as a set of Docker containers managed by docker-compose.

The services are defined in the various `docker-compose*.yaml` which can be found in the `~/wis2box/` directory.

The Python script `wis2box-ctl.py` is used to run the underlying Docker Compose commands that control the wis2box services.

You don't need to know the details of the Docker containers to run the wis2box software stack, but you can inspect the `docker-compose*.yaml` and files to see how the services are defined. If you are interested in learning more about Docker, you can find more information in the [Docker documentation](#).

To login to the wis2box-management container, use the following command:

```
python3 wis2box-ctl.py login
```

Inside the wis2box-management container you can run various commands to manage your wis2box, such as:

- `wis2box auth add-token --path processes/wis2box` : to create an authorization token for the `processes/wis2box` endpoint
- `wis2box data clean --days=<number-of-days>` : to clean up data older than a certain number of days from the `wis2box-public` bucket

To exit the container and go back to the host machine, use the following command:

```
exit
```

Run the following command to see the docker containers running on your host machine:

```
docker ps
```

You should see the following containers running:

- wis2box-management
- wis2box-api
- wis2box-minio
- wis2box-webapp
- wis2box-auth
- wis2box-ui
- wis2downloader
- elasticsearch
- elasticsearch-exporter
- nginx
- mosquitto
- prometheus
- grafana
- loki

These containers are part of the wis2box software stack and provide the various services required to run the wis2box.

Run the following command to see the docker volumes running on your host machine:

```
docker volume ls
```

You should see the following volumes:

- wis2box_project_auth-data
- wis2box_project_es-data
- wis2box_project_htpasswd
- wis2box_project_minio-data
- wis2box_project_prometheus-data
- wis2box_project_loki-data

- wis2box_project_mosquitto-config

As well as some anonymous volumes used by the various containers.

The volumes starting with `wis2box_project_` are used to store persistent data for the various services in the wis2box software stack.

2.3.4 wis2box API

The wis2box contains an API (Application Programming Interface) that provide data access and processes for interactive visualization, data transformation and publication.

Open a new tab and navigate to the page `http://YOUR-HOST/oapi`.


[Contact](#)

[Home](#)
[json](#)
[jsonld](#)

WIS2 in a box API

WIS 2.0 in a box (wis2box) provides a platform with the capabilities for centres to integrate their data holdings and publish them to the WMO Information System with a plug and play capability supporting data publishing, discovery and access. wis2box-api provides an API providing data and processes for interactive visualization, data transformation and publication.

[wis2box](#)
[WIS2 Node](#)

Terms of service

License

<https://www.apache.org/licenses/LICENSE-2.0>
[Apache License 2.0](#)

Collections

[View the collections in this service](#)

Processes

[View the processes in this service](#)

Provider

WIS 2.0 in a box
<https://docs.wis2box.wis.wmo.int>

Contact point

Address
 7bis, avenue de la Paix
 Geneva,
 1211
 Switzerland
Email

Contact URL
<https://wmo.int/contact-us>

This is the landing page of the wis2box API (running via the **wis2box-api** container).

? Question

What collections are currently available?

✓ Click to reveal answer



To view collections currently available through the API, click `View the collections in this service`:

Data collections in this service

Name	Description
Stations	Stations
Data notifications	Data notifications

Record collections in this service

Name	Description
Discovery metadata	Discovery metadata

The following collections are currently available:

- Stations
- Data notifications
- Discovery metadata

? Question

How many data notifications have been published?

✓ Click to reveal answer



Click on "Data notifications", then click on `Browse through the items of "Data Notifications"`.

You will note that the page says "No items" as no Data notifications have been published yet.

2.3.5 wis2box webapp

Open a web browser and visit the page `http://YOUR-HOST/wis2box-webapp`.

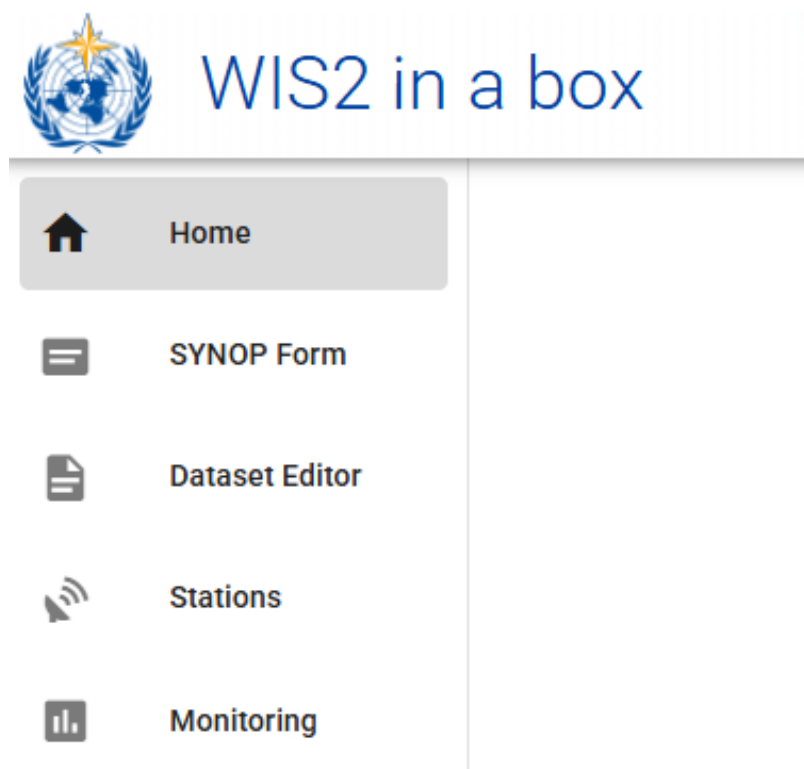
You will see a pop-up asking for your username and password. Use the default username `wis2box-user` and the `WIS2BOX_WEBAPP_PASSWORD` defined in the `wis2box.env` file and click "Sign in":

 Note

Check your `wis2box.env` for the value of your `WIS2BOX_WEBAPP_PASSWORD`. You can use the following command to check the value of this environment variable:

```
cat ~/wis2box/wis2box.env | grep WIS2BOX_WEBAPP_PASSWORD
```

Once logged in, you move your mouse to the menu on the left to see the options available in the wis2box web application:



This is the wis2box web application to enable you to interact with your wis2box:

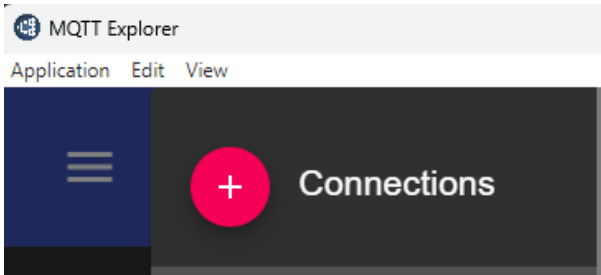
- create and manage datasets
- update/review your station metadata
- upload manual observations using FM-12 synop form
- monitor notifications published on your wis2box-broker

We will use this web application in a later session.

2.3.6 wis2box-broker

Open the MQTT Explorer on your computer and prepare a new connection to connect to your broker (running via the **wis2box-broker** container).

Click **+** to add a new connection:



You can click on the 'ADVANCED' button and verify you have subscriptions to the the following topics:

- `#`
- `$SYS/#`

Topic		QoS	+ ADD
		0 ▼	
Topic		QoS	
	#	0	
	\$SYS/#	0	

Note

The `#` topic is a wildcard subscription that will subscribe to all topics published on the broker.

The messages published under the `$SYS` topic are system messages published by the mosquitto service itself.

Use the following connection details, making sure to replace the value of `<your-host>` with your hostname and `<WIS2BOX_BROKER_PASSWORD>` with the value from your `wis2box.env` file:

- **Protocol:** `mqtt://`
- **Host:** `<your-host>`
- **Port:** `1883`
- **Username:** `wis2box`
- **Password:** `<WIS2BOX_BROKER_PASSWORD>`

 Note

You can check your `wis2box.env` for the value of your `WIS2BOX_BROKER_PASSWORD`. You can use the following command to check the value of this environment variable:

```
cat ~/wis2box/wis2box.env | grep WIS2BOX_BROKER_PASSWORD
```

Note that this your **internal** broker password, the Global Broker will use different (read-only) credentials to subscribe to your broker. Never share this password with anyone.

Make sure to click "SAVE" to store your connection details.

Then click "CONNECT" to connect to your **wis2box-broker**.

MQTT Connection

mqtt://mlimper.wis2.training:1883/

Name

mlimper.wis2.training

☐

Validate certificate

☐

Encryption (tls)

Protocol

mqtt://

▼

Host

mlimper.wis2.training

Port

1883

Username

wis2box

Password

.....



DELETE 

 ADVANCED

 SAVE

 CONNECT

Once you are connected, verify that your the internal mosquitto statistics being published by your broker under the `$$SYS` topic:

▼ \$SYS

▼ broker

version = mosquitto version 2.0.20

uptime = 1276 seconds

▶ **clients** (7 topics, 10 messages)

▶ **load** (27 topics, 169 messages)

▶ **messages** (3 topics, 19 messages)

▶ **store** (2 topics, 6 messages)

▶ **subscriptions** (1 topic, 2 messages)

▶ **shared_subscriptions** (1 topic, 1 message)

▶ **retained messages** (1 topic, 1 message)

▶ **publish** (5 topics, 21 messages)

▶ **bytes** (2 topics, 18 messages)

Keep the MQTT Explorer open, as we will use it to monitor the messages published on the broker.

2.3.7 Conclusion

✓ Congratulations!

In this practical session, you learned how to:

- run the `wis2box-create-config.py` script to create the initial configuration
- start wis2box and check the status of its components
- access the wis2box-webapp and wis2box-API in a browser
- connect to the MQTT broker on your student VM using MQTT Explorer

2.4 Configuring datasets in wis2box



Learning outcomes

By the end of this practical session, you will be able to:

- create a new dataset
- create discovery metadata for a dataset
- configure data mappings for a dataset
- publish a WIS2 notification with a WCMP2 record
- update and re-publish your dataset

2.4.1 Introduction

wis2box uses datasets that are associated with discovery metadata and data mappings.

Discovery metadata is used to create a WCMP2 (WMO Core Metadata Profile 2) record that is shared using a WIS2 notification published on your wis2box-broker.

The data mappings are used to associate a data plugin to your input data, allowing your data to be transformed prior to being published using the WIS2 notification.

This session will walk you through creating a new dataset, creating discovery metadata, and configuring data mappings. You will inspect your dataset in the wis2box-api and review the WIS2 notification for your discovery metadata.

2.4.2 Preparation

Connect to your broker using MQTT Explorer.

Instead of using your internal broker credentials, use the public credentials `everyone/everyone` :

MQTT Connection

mqtt://mlimper.wis2.training:1883/

Name

mlimper.wis2.training

☐

Validate certificate

☐

Encryption (tls)

Protocol

mqtt://

Host

mlimper.wis2.training

Port

1883

Username

everyone

Password

everyone

☐

DELETE

ADVANCED

SAVE

CONNECT

Note

You never need to share the credentials of your internal broker with external users. The 'everyone' user is a public user to enable sharing of WIS2 notifications.

The `everyone/everyone` credentials has read-only access on the topic 'origin/a/wis2/#'. This is the topic where the WIS2 notifications are published. The Global Broker can subscribe with these public credentials to receive the notifications.

The 'everyone' user will not see internal topics or be able to publish messages.

Open a browser and open a page to `http://YOUR-HOST/wis2box-webapp`. Make sure you are logged in and can access the 'dataset editor' page.

See the section on [Initializing wis2box](#) if you need to remember how to connect to the broker or access the wis2box-webapp.

2.4.3 Create an authorization token for processes/wis2box

You will need an authorization token for the 'processes/wis2box' endpoint to publish your dataset.

To create an authorization token, access your training VM over SSH and use the following commands to login to the wis2box-management container:

```
cd ~/wis2box
python3 wis2box-ctl.py login
```

Then run the following command to create a randomly generated authorization token for the 'processes/wis2box' endpoint:

```
wis2box auth add-token --path processes/wis2box
```

You can also create a token with a specific value by providing the token as an argument to the command:

```
wis2box auth add-token --path processes/wis2box MyS3cretToken
```

Make sure to copy the token value and store it on your local machine, as you will need it later.

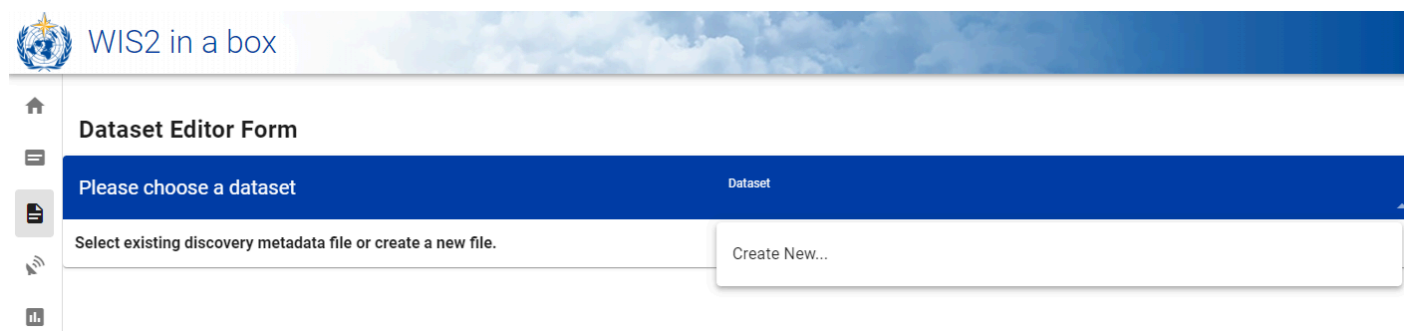
Once you have your token, you can exit the wis2box-management container:

```
exit
```

2.4.4 Creating a new dataset in the wis2box-webapp

Navigate to the 'dataset editor' page in the wis2box-webapp of your wis2box instance by going to `http://YOUR-HOST/wis2box-webapp` and selecting 'dataset editor' from the menu on the left hand side.

On the 'dataset editor' page, under the 'Datasets' tab, click on "Create New ...":



A pop-up window will appear, asking you to provide:

- **Centre ID** : this is the agency acronym (in lower case and no spaces), as specified by the WMO Member, that identifies the data centre responsible for publishing the data.
- **Data Type**: The type of data you are creating metadata for. You can choose between using a predefined template or selecting 'other'. If 'other' is selected, more fields will have to be manually filled.

Centre ID

Your centre-id should start with the TLD of your country, followed by a dash (-) and an abbreviated name of your organization (for example `fr-meteofrance`). The centre-id must be lowercase and use alphanumeric characters only. The dropdown list shows all currently registered centre-ids on WIS2 as well as any centre-id you have already created in wis2box.

Data Type Templates

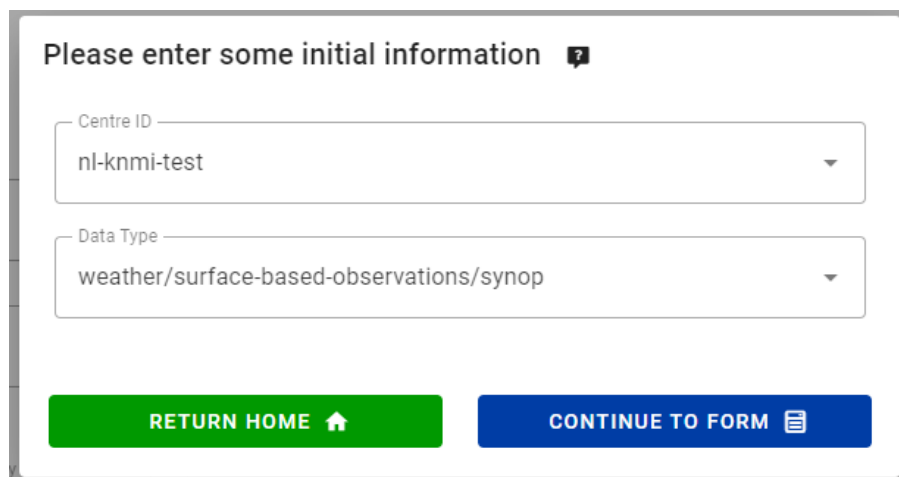
The *Data Type* field allows you to select from a list of templates available in the wis2box-webapp dataset editor. A template will pre-populate the form with suggested default values appropriate for the data type. This includes suggested title and keywords for the metadata and pre-configured data plugins. The topic will be fixed to the default topic for the data type.

For the purpose of the training we will use the *weather/surface-based-observations/synop* data type which includes data plugins that ensure the data is transformed into BUFR format before being published.

If you want to publish CAP alerts using wis2box, use the template *weather/advisories-warnings*. This template includes a data plugin that verifies the input data is a valid CAP alert before publishing. To create CAP alerts and publish them via wis2box you can use [CAP Composer](#).

Please choose a centre-id appropriate for your organization.

For **Data Type**, select **weather/surface-based-observations/synop**:



Please enter some initial information 

Centre ID
nl-knmi-test

Data Type
weather/surface-based-observations/synop

RETURN HOME  **CONTINUE TO FORM** 

Click *continue to form* to proceed, you will now be presented with the **Dataset Editor Form**.

Since you selected the **weather/surface-based-observations/synop** data type, the form will be pre-populated with some initial values related to this data type.

2.4.5 Creating discovery metadata

The Dataset Editor Form allows you to provide the Discovery Metadata for your dataset that the wis2box-management container will use to publish a WCMP2 record.

Since you have selected the 'weather/surface-based-observations/synop' data type, the form will be pre-populated with some default values.

Please make sure to replace the auto-generated 'Local ID' with a descriptive name for your dataset, e.g. 'synop-dataset-wis2training':

Dataset Identification 

Title

Hourly synoptic observations from fixed-land stations

Local ID

synop-dataset-wis2-training

Identifier

urn:wmo:md:nl-knmi-test:synop-dataset-wis2-training



Centre ID

nl-knmi-te

WMO Data Policy

core

▼

Topic Hierarchy

nl-knmi-test/data/core/weather/surface-based-observations/synop

Earth System Disciplines

Weather

▼

Keywords (3 minimu...

+

observations

×

temperature

×

visibility

×

Review the title and keywords, and update them as necessary, and provide a description for your dataset.

Note there are options to change the 'WMO Data Policy' from 'core' to 'recommended' or to modify your default Metadata Identifier, please keep data-policy as 'core' and use the default Metadata Identifier.

Next, review the section defining your 'Temporal Properties' and 'Spatial Properties'. You can adjust the bounding box by updating the 'North Latitude', 'South Latitude', 'East Longitude', and 'West Longitude' fields:

Page 34

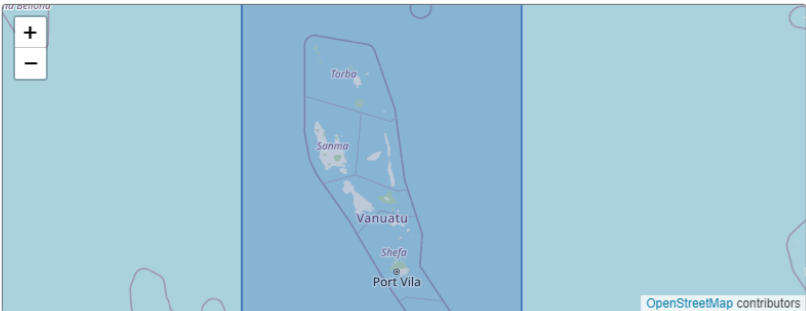
Temporal Properties ⓘ

2024-05-30 × End Date in UTC ☒ Dataset ongoing Resolution 1 Unit hour(s) ▼

Spatial Properties ⓘ

Choose an automatic bounding box (optional) ▼
Your country may not have an automatic bounding box

West Longitude 165 North Latitude -12 East Longitude 171 South Latitude -21



Next, fill out the section defining the 'Contact Information of the Data Provider':

Contact Information of the Data Provider ⓘ

Organization Name WMO URL https://wmo.int Country Switzerland ▼

Email wis2-support@wmo.int × Phone number (optional) 

Finally, fill out the section defining the 'Data Quality Information':

Once you are done filling out all the sections, click 'VALIDATE FORM' and check the form for any errors:

Contact Information of the Data Provider ⓘ

Organization Name Field is required URL https://wmo.int Country Switzerland ▼

Email wis2-support@wmo.int

Validation Outcome

Form is invalid, please check all of the fields are filled correctly and try again.

OK

RESET FORM VALIDATE FORM ✓

If there are any errors, correct them and click 'VALIDATE FORM' again.

Making sure you have no errors and that you get a pop-up indication your form has been validated:

Contact Information of the Data Provider ⓘ

Organization Name WMO URL URL Country Switzerland ▼

Email wis2-support@wmo.int

Validation Outcome

Form is valid, please proceed.

OK

RESET FORM VALIDATE FORM ✓

Next, before submitting your dataset, review the data mappings for your dataset.

2.4.6 Configuring data mappings

Since you used a template to create your dataset, the dataset mappings have been pre-populated with the defaults plugins for the 'weather/surface-based-observations/synop' data type. Data plugins are used in the wis2box to transform data before it is published using the WIS2 notification.

Dataset Mappings Editor

Plugins in use	File extension	File pattern		
BUFR data converted to BUFR	bin	^.*\.bin\$	UPDATE	DELETE
FM-12 data converted to BUFR	txt	^.*_(\d{4})(\d{2}).*txt\$	UPDATE	DELETE
BUFR data converted to BUFR	b	^.*\.b\$	UPDATE	DELETE
CSV data converted to BUFR	csv	^.*\.csv\$	UPDATE	DELETE
BUFR data converted to GeoJSON	bufr4	^WIGOS_(\d-\d+-\d+-\w+)_.*\.bufr4\$	UPDATE	DELETE

[ADD A PLUGIN](#) +

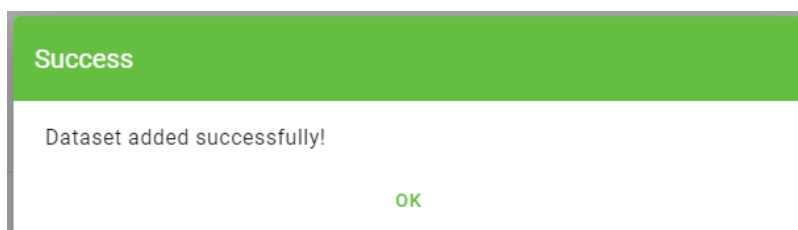
Note that you can click on the "update"-button to change settings for the plugin such as file-extension and the file-pattern, you can leave the default settings for now. In a later session, you will learn more about BUFR and the transformation of data into BUFR format.

2.4.7 Submitting your dataset

Finally, you can click 'submit' to publish your dataset.

You will need to provide the authorization token for 'processes/wis2box' that you created earlier. If you have not done so, you can create a new token by following the instructions in the preparation section.

Check that you get the following message after submitting your dataset, indicating that the dataset was successfully submitted:



After you click 'OK', you are redirected to the Dataset Editor home page. Now if you click on the 'Dataset' tab, you should see your new dataset listed:

Dataset Editor Form

Please choose a dataset

Dataset

Select existing discovery metadata file or create a new file.

urn:wmo.md:nl-knmi-test:surface-based-observations.synop

Create New...

2.4.8 Reviewing the WIS2-notification for your discovery metadata

Go to MQTT Explorer, if you were connected to the broker, you should see a new WIS2 notification published on the topic `origin/a/wis2/<your-centre-id>/metadata`:

The screenshot shows the MQTT Explorer interface. On the left, a tree view shows the topic structure: `localhost` > `origin` > `a` > `wis2` > `nl-knmi-test`. The selected topic is `nl-knmi-test`, and its value is `metadata = {"id": "bae0ab76-bf6d-461a-af5f-8a635a81b365",`. On the right, the 'Value' panel displays the JSON message:

```
{  "id": "bae0ab76-bf6d-461a-af5f-8a635a81b365",  "type": "Feature",  "version": "v04",  "geometry": {    "type": "Polygon",    "coordinates": [      [        [165, -12]      ]    ]  }}
```

. The 'History' panel at the bottom shows 1 message. The top right of the interface shows 'DISCONNECT' and a user icon.

Inspect the content of the WIS2 notification you published. You should see a JSON with a structure corresponding to the WIS Notification Message (WNM) format.

? Question

On what topic is the WIS2 notification published?

✓ Click to reveal answer

The WIS2 notification is published on the topic `origin/a/wis2/<your-centre-id>/metadata`.

? Question

Try to find the title, description and keywords you provided in the discovery metadata in the WIS2 notification. Can you find them?

✓ Click to reveal answer

The title, description, and keywords you provided in the discovery metadata are not present in the WIS2 notification payload!

Instead, try to look for the canonical link in the "links"-section in the WIS2 notification:

```
"links": [
  {
    "rel": "canonical",
    "type": "application/geo+json",
    "href": "http://136.156.130.125/data/metadata/urn:wmo:md:nl-knmi-test:core.surface-based-observations.synop.json",
    "length": 1753
  }
],
```

The WIS2 notification contains a canonical link to the WCMP2 record that was published.

Copy-paste this canonical link into your browser to access the WCMP2 record, depending on your browser settings, you may be prompted to download the file or it may be displayed directly in your browser.

You will find the title, description, and keywords you provided inside the WCMP2 record.

2.4.9 Conclusion

✓ Congratulations!

In this practical session, you learned how to:

- create a new dataset
- define your discovery metadata
- review your data mappings
- publish discovery metadata
- review the WIS2 notification for your discovery metadata

2.5 Configuring station metadata

Learning outcomes

By the end of this practical session, you will be able to:

- create an authorization token for the `collections/stations` endpoint
- add station metadata to wis2box
- update/delete station metadata using the **wis2box-webapp**

2.5.1 Introduction

For sharing data internationally between WMO Members, it is important to have a common understanding of the stations that are producing the data. The WMO Integrated Global Observing System (WIGOS) provides a framework for the integration of observing systems and data management systems. The **WIGOS Station Identifier (WSI)** is used as the unique reference of the station which produced a specific set of observation data.

wis2box has a collection of station metadata that is used to describe the stations that are producing the observation data and should be retrieved from **OSCAR/Surface**. The station metadata in wis2box is used by the BUFR transformation tools to check that input data contains a valid WIGOS Station Identifier (WSI) and to provide a mapping between the WSI and the station metadata.

2.5.2 Create an authorization token for collections/stations

To edit stations via the **wis2box-webapp** you will first to need create an authorization token.

Login to your student VM and ensure you are in the `wis2box` directory:

```
cd ~/wis2box
```

Then login into the **wis2box-management** container with the following command:

```
python3 wis2box-ctl.py login
```

Within the **wis2box-management** container your can create an authorization token for a specific endpoint using the command: `wis2box auth add-token --path <my-endpoint>`.

For example, to use a random automatically generated token for the `collections/stations` endpoint:

```
wis2box auth add-token --path collections/stations
```

The output will look like this:

```
Continue with token: 7ca20386a131f0de384e6ffa288eb1ae385364b3694e47e3b451598c82e899d1 [y/N]? y
Token successfully created
```

Or, if you want to define your own token for the `collections/stations` endpoint, you can use the following example:

```
wis2box auth add-token --path collections/stations DataIsMagic
```

Output:

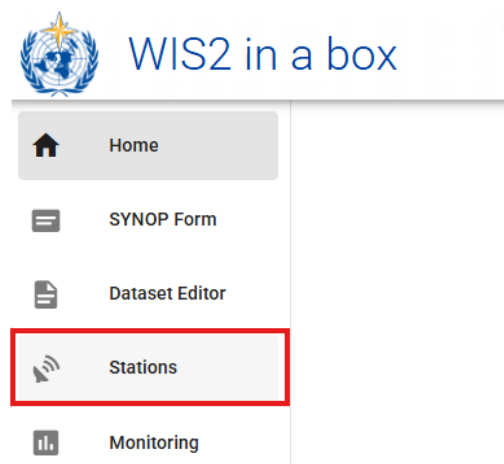
```
Continue with token: DataIsMagic [y/N]? y
Token successfully created
```

Please create an authorization token for the `collections/stations` endpoint using the instructions above.

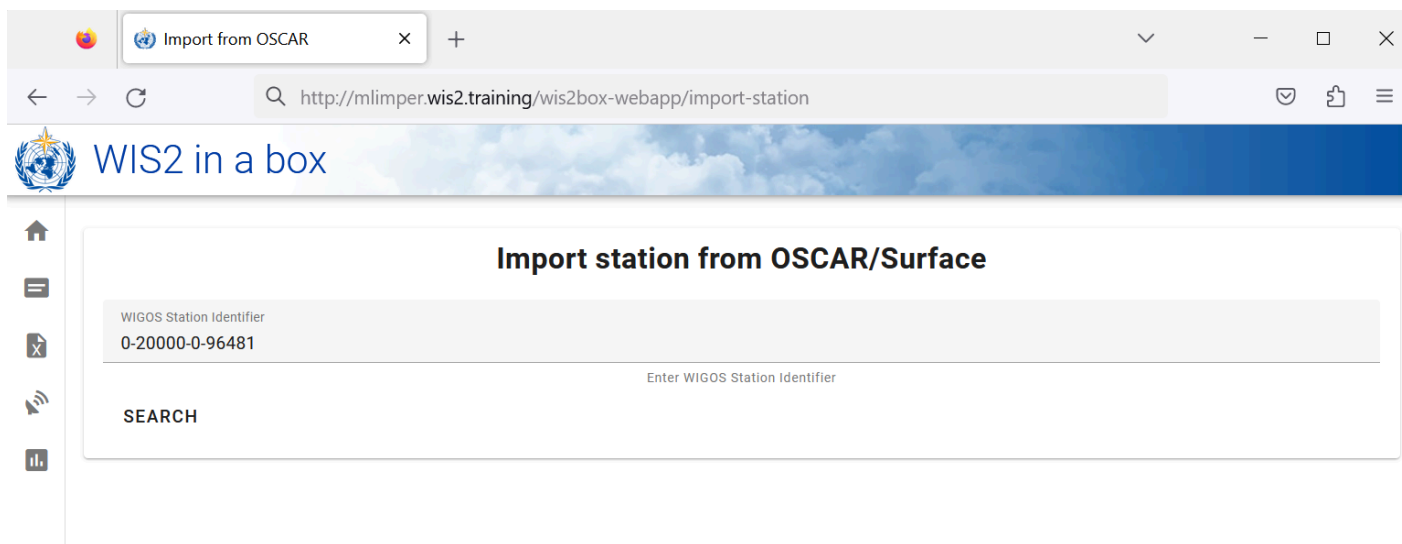
2.5.3 add station metadata using the **wis2box-webapp**

The **wis2box-webapp** provides a graphical user interface to edit station metadata.

Open the **wis2box-webapp** in your browser by navigating to `http://YOUR-HOST/wis2box-webapp`, and select stations:



When you click add 'add new station' you are asked to provide the WIGOS station identifier for the station you want to add:



The screenshot shows a web browser window with the address bar displaying `http://mlimper.wis2.training/wis2box-webapp/import-station`. The page header features the WIS2 in a box logo and title. A sidebar on the left contains navigation icons. The main content area is titled "Import station from OSCAR/Surface" and contains a form with a text input field labeled "WIGOS Station Identifier" containing the value "0-20000-0-96481". Below the input field is a "SEARCH" button. A placeholder text "Enter WIGOS Station Identifier" is visible below the input field.

Add station metadata for 3 or more stations

Please add three or more stations to the wis2box station metadata collection of your wis2box.

Please use stations from your country if possible, especially if you brought your own data.

If your country does not have any stations in OSCAR/Surface, you can use the following stations for the purpose of this exercise:

- 0-20000-0-91334
- 0-20000-0-96323 (note missing station elevation in OSCAR)
- 0-20000-0-96749 (note missing station elevation in OSCAR)

When you click search the station data is retrieved from OSCAR/Surface, please note that this can take a few seconds.

Review the data returned by OSCAR/Surface and add missing data where required. Select a topic for the station and provide your authorization token for the `collections/stations` endpoint and click 'save':


WIS2 in a box







Facility type
Land (fixed)

Station/platform on solid terrain, at fixed position

Barometer height above sea level
 20

Enter barometer height (metres)

WMO Region
South-West Pacific

South-West Pacific

Territory or WMO member operating the station
Malaysia

Malaysia

Operating status
Operational

The station fully complies with the reporting obligations of the observation programme/network concerned.

Topic hierarchy
 origin/a/wis2/mys/maaike_test/data/core/weather/surface-based-observations/synop

wis2box auth token for "collections/stations"

SAVE CANCEL

Station successfully submitted, redirecting to station viewer.

CLOSE

Go back to the station list and you will see the station you added:


WIS2 in a box







Stations

actions	id	station_name	wigos_station_identifier	traditional_station_identifier	facility_type	latitude	longitude
  	0-20000-0-96481	TAWAU	0-20000-0-96481	96481	Land (fixed)	4.2666666667	117.8833333333

Items per page: 10 1-1 of 1

+ ADD NEW STATION

Repeat this process until you have at least 3 stations configured.



Deriving missing elevation information

If your station elevation is missing, there are online services to help lookup the elevation using open elevation data. One such example is the [Open Topo Data API](#).

For example, to get the elevation at latitude -6.15558 and longitude 106.84204, you can copy-paste the following URL in a new browser-tab:

```
https://api.opentopodata.org/v1/aster30m?locations=-6.15558,106.84204
```

Output:

```
{
  "results": [
    {
      "dataset": "aster30m",
      "elevation": 7.0,
      "location": {
        "lat": -6.15558,
        "lng": 106.84204
      }
    }
  ],
  "status": "OK"
}
```

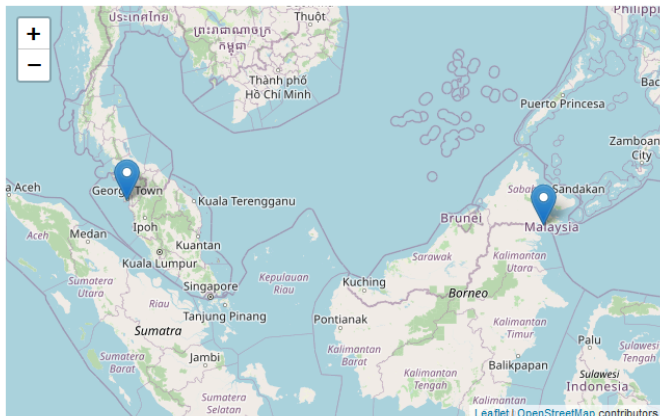
2.5.4 Review your station metadata

The station metadata is stored in the backend of wis2box and made available via the **wis2box-api**.

If you open a browser and navigate to `http://YOUR-HOST/oapi/collections/stations/items` you will see the station metadata you added:

Stations

Items in this collection.



Warning: Higher limits not recommended!

Limit: 500 (default) ▼

wigos_station_identifier	name	traditional_station_identifier	f
0-20000-0-96481	TAWAU	96481	L
0-20000-0-48601	PENANG/BAYAN LEPAS	48601	L

Review your station metadata

Verify the stations you added are associated to your dataset by visiting `http://YOUR-HOST/oapi/collections/stations/items` in your browser.

You also have the option to view/update/delete the station in the **wis2box-webapp**. Note that you are required to provide your authorization token for the `collections/stations` endpoint to update/delete the station.

Update/delete station metadata

Try and see if you can update/delete the station metadata for one of the stations you added using the **wis2box-webapp**.

2.5.5 Bulk station metadata upload

Note that wis2box also has the ability to perform "bulk" loading of station metadata from a CSV file using the command line in the **wis2box-management** container.

```
python3 wis2box-ctl.py login
wis2box metadata station publish-collection -p /data/wis2box/metadata/station/station_list.csv -
th origin/a/wis2/centre-id/weather/surface-based-observations/synop
```

This allows you to upload a large number of stations at once and associate them with a specific topic.

You can create the CSV file using Excel or a text editor and then upload it to `wis2box-host-datadir` to make it available to the **wis2box-management** container in the `/data/wis2box/` directory.

After doing a bulk upload of stations, it is recommended to review the stations in the **wis2box-webapp** to ensure the data was uploaded correctly.

See the official [wis2box documentation](#) for more information on how to use this feature.

2.5.6 Conclusion

✓ | Congratulations!

In this practical session, you learned how to:

- create an authorization token for the `collections/stations` endpoint to be used with the **wis2box-webapp**
- add station metadata to wis2box using the **wis2box-webapp**
- view/update/delete station metadata using the **wis2box-webapp**

2.6 Ingesting data for publication

Learning Outcomes

By the end of this practical session, you will be able to:

- Trigger the wis2box workflow by uploading data to MinIO using the command line, the MinIO web interface, SFTP, or a Python script.
- Access the Grafana dashboard to monitor the status of data ingestion and view logs of your wis2box instance.
- View WIS2 data notifications published by your wis2box using MQTT Explorer.

2.6.1 Introduction

In WIS2, data is shared in real-time using WIS2 data notifications that contain a "canonical" link from which the data can be downloaded.

To trigger the data workflow in a WIS2 Node using the wis2box software, data must be uploaded to the **wis2box-incoming** bucket in **MinIO**, which initiates the wis2box workflow. This process results in the data being published via a WIS2 data notification. Depending on the data mappings configured in your wis2box instance, the data may be transformed into BUFR format before being published.

In this exercise, we will use sample data files to trigger the wis2box workflow and **publish WIS2 data notifications** for the dataset you configured in the previous practical session.

During the exercise, we will monitor the status of the data ingestion using the **Grafana dashboard** and **MQTT Explorer**. The Grafana dashboard uses data from Prometheus and Loki to display the status of your wis2box, while MQTT Explorer allows you to see the WIS2 data notifications published by your wis2box instance.

Note that wis2box will transform the example data into BUFR format before publishing it to the MQTT broker, as per the data mappings pre-configured in your dataset. For this exercise, we will focus on the different methods to upload data to your wis2box instance and verify successful ingestion and publication. Data transformation will be covered later in the [Data Conversion Tools](#) practical session.

2.6.2 Preparation

This section uses the dataset for "surface-based-observations/synop" previously created in the [Configuring Datasets in wis2box](#) practical session. It also requires knowledge of configuring stations in the **wis2box-webapp**, as described in the [Configuring Station Metadata](#) practical session.

Ensure you can log in to your student VM using your SSH client (e.g., PuTTY).

Ensure wis2box is up and running:

```
cd ~/wis2box/  
python3 wis2box-ctl.py start  
python3 wis2box-ctl.py status
```

Ensure MQTT Explorer is running and connected to your instance using the public credentials `everyone/everyone` with a subscription to the topic `origin/a/wis2/#`.

Ensure you have a web browser open with the Grafana dashboard for your instance by navigating to `http://YOUR-HOST:3000`.

Prepare Example Data

Copy the directory `exercise-materials/data-ingest-exercises` to the directory you defined as the `WIS2BOX_HOST_DATADIR` in your `wis2box.env` file:

```
cp -r ~/exercise-materials/data-ingest-exercises ~/wis2box-data/
```

Note

The `WIS2BOX_HOST_DATADIR` is mounted as `/data/wis2box/` inside the `wis2box-management` container by the `docker-compose.yml` file included in the `wis2box` directory.

This allows you to share data between the host and the container.

Add the Test Station

Add the station with WIGOS identifier `0-20000-0-64400` to your `wis2box` instance using the station editor in the `wis2box-webapp`.

Retrieve the station from OSCAR:

Import station from OSCAR/Surface

WIGOS Station Identifier

0-20000-0-64400

Enter WIGOS Station Identifier

SEARCH

Add the station to the datasets you created for publishing on `"../surface-based-observations/synop"` and save the changes using your authentication token:

Topics

origin/a/wis2/my-centre-id/data/core/weather/surface-based-observations/synop ▼

wis2box auth token for "collections/stations"

.....

SAVE

CANCEL

Note that you can remove this station from your dataset after the practical session.

2.6.3 Testing Data Ingestion from the Command Line

In this exercise, we will use the `wis2box data ingest` command to upload data to MinIO.

Ensure you are in the `wis2box` directory and log in to the **wis2box-management** container:

```
cd ~/wis2box
python3 wis2box-ctl.py login
```

Verify the following sample data is available in the directory `/data/wis2box/` within the **wis2box-management** container:

```
ls -lh /data/wis2box/data-ingest-exercises/synop_202412030900.txt
```

? Ingesting Data Using `wis2box data ingest`

Execute the following command to ingest the sample data file into your wis2box instance:

```
wis2box data ingest -p /data/wis2box/data-ingest-exercises/synop_202412030900.txt --metadata-id
urn:wmo:md:not-my-centre:synop-test
```

Was the data successfully ingested? If not, what was the error message, and how can you fix it?

 Click to Reveal Answer

The data was **not** successfully ingested. You should see the following:

```
Error: metadata_id=urn:wmo:md:not-my-centre:synop-test not found in data mappings
```

The error message indicates that the metadata identifier you provided does not match any of the datasets you have configured in your wis2box instance.

Provide the correct metadata ID that matches the dataset you created in the previous practical session and repeat the data ingestion command until you see the following output:

```
Processing /data/wis2box/data-ingest-exercises/synop_202412030900.txt
Done
```

Go to the MinIO console in your browser and check if the file `synop_202412030900.txt` was uploaded to the `wis2box-incoming` bucket. You should see a new directory with the name of the dataset you provided in the `--metadata-id` option, and inside this directory, you will find the file `synop_202412030900.txt`:

**wis2box-incoming**
Created on: **Mon, Sep 30 2024 15:17:57**
(GMT+2) Access: **PRIVATE** 52.9 KiB - 1 Object

Rewind ↺ Refresh ↻ Upload ↗

< wis2box-incoming / urn:wmo:md:my-centre-id:synop-test  Create new path ://

☐	▲ Name	Last Modified	Size
☐	 synop_202412030900.txt	Today, 11:27	122.0 B

 Note

The `wis2box data ingest` command uploaded the file to the `wis2box-incoming` bucket in MinIO in a directory named after the metadata identifier you provided.

Go to the Grafana dashboard in your browser and check the status of the data ingestion.

 Check the Status of the Data Ingestion on Grafana

Go to the Grafana dashboard at **`http://your-host:3000`** and check the status of the data ingestion in your browser.

How can you see if the data was successfully ingested and published?

 Click to Reveal Answer

If you successfully ingested the data, you should see the following:

Station data publishing status		
WSI	Publish last 24 hrs	Fail last 24 hrs
0-20000-0-64400	1	0

If you do not see this, please check for WARNING or ERROR messages displayed at the bottom of the dashboard and attempt to resolve them.

 Check the MQTT Broker for WIS2 Notifications

Go to the MQTT Explorer and check if you can see the WIS2 notification message for the data you just ingested.

How many WIS2 data notifications were published by your wis2box?

How do you access the content of the data being published?

 Click to Reveal Answer

You should see 1 WIS2 data notification published by your wis2box.

To access the content of the data being published, you can expand the topic structure to see the different levels of the message until you reach the last level and review the message content.

The message content has a "links" section with a "rel" key of "canonical" and a "href" key with the URL to download the data. The URL will be in the format `http://YOUR-HOST/data/...`

Note that the data format is BUFR, and you will need a BUFR parser to view the content of the data. The BUFR format is a binary format used by meteorological services to exchange data. The data plugins inside wis2box transformed the data into BUFR before publishing it.

After completing this exercise, exit the **wis2box-management** container:

```
exit
```

2.6.4 Uploading Data Using the MinIO Web Interface

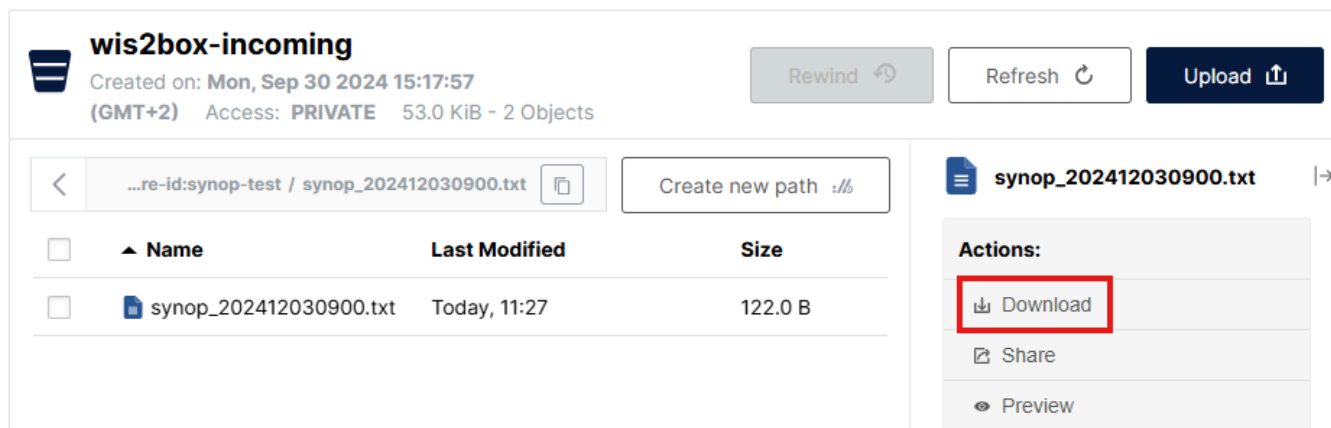
In the previous exercises, you uploaded data available on the wis2box host to MinIO using the `wis2box data ingest` command.

Next, we will use the MinIO web interface, which allows you to download and upload data to MinIO using a web browser.

? Re-upload Data Using the MinIO Web Interface

Go to the MinIO web interface in your browser and browse to the `wis2box-incoming` bucket. You will see the file `synop_202412030900.txt` you uploaded in the previous exercises.

Click on the file, and you will have the option to download it:



You can download this file and re-upload it to the same path in MinIO to re-trigger the wis2box workflow.

Check the Grafana dashboard and MQTT Explorer to see if the data was successfully ingested and published.

✓ Click to Reveal Answer

You will see a message indicating that the wis2box already published this data:

```
ERROR - Data already published for WIGOS_0-20000-0-64400_20241203T090000-bufr4; not publishing
```

This demonstrates that the data workflow was triggered, but the data was not re-published. The wis2box will not publish the same data twice.

? Upload New Data Using the MinIO Web Interface

Download this sample file `synop_202502040900.txt` (right-click and select "save as" to download the file).

Upload the file you downloaded using the web interface to the same path in MinIO as the previous file.

Did the data ingest and publish successfully?

✓ Click to Reveal Answer

Go to the Grafana dashboard and check if the data was successfully ingested and published.

If you use the wrong path, you will see an error message in the logs.

If you use the correct path, you will see one more WIS2 data notification published for test station `0-20000-0-64400`, indicating that the data was successfully ingested and published.

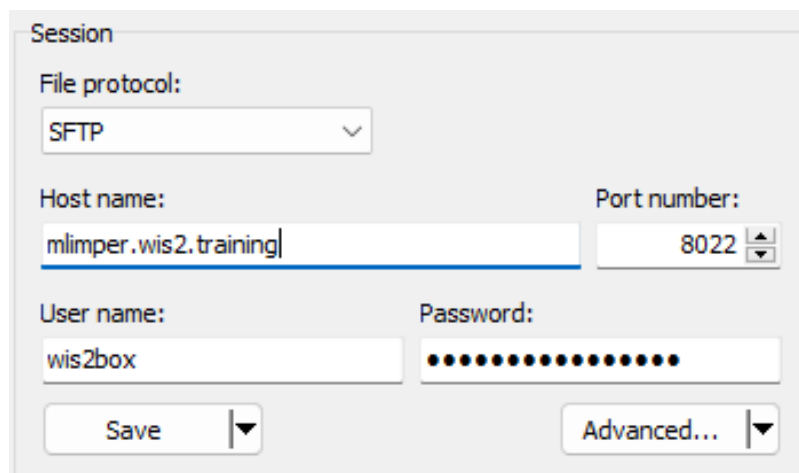
Station data publishing status		
WSI	Publish last 24 hrs	Fail last 24 hrs
0-20000-0-64400	2	0

2.6.5 Uploading Data Using SFTP

The MinIO service in wis2box can also be accessed over SFTP. The SFTP server for MinIO is bound to port 8022 on the host (port 22 is used for SSH).

In this exercise, we will demonstrate how to use WinSCP to upload data to MinIO using SFTP.

You can set up a new WinSCP connection as shown in this screenshot:



The screenshot shows the WinSCP 'Session' configuration dialog. The 'File protocol' is set to 'SFTP'. The 'Host name' is 'mlimper.wis2.training' and the 'Port number' is '8022'. The 'User name' is 'wis2box' and the 'Password' field is masked with dots. There are 'Save' and 'Advanced...' buttons at the bottom.

The credentials for the SFTP connection are defined by `WIS2BOX_STORAGE_USERNAME` and `WIS2BOX_STORAGE_PASSWORD` in your `wis2box.env` file and are the same as the credentials you used to connect to the MinIO UI.

When you log in, you will see the buckets used by wis2box in MinIO:

/		
Name	Size	Changed
..		
wis2box-public		9/30/2024 3:17:58 PM
wis2box-incoming		9/30/2024 3:17:57 PM

You can navigate to the `wis2box-incoming` bucket and then to the folder for your dataset. You will see the files you uploaded in the previous exercises:

/wis2box-incoming/urn:wmo:md:my-centre-id:synop-test/		
Name	Size	Changed
..		
synop_202412030900.txt	1 KB	4/2/2025 1:34:20 PM
synop_202502040900.txt	1 KB	4/2/2025 1:31:06 PM

? Upload Data Using SFTP

Download this sample file to your local computer:

[synop_202503030900.txt](#) (right-click and select "save as" to download the file).

Then upload it to the incoming dataset path in MinIO using your SFTP session in WinSCP.

Check the Grafana dashboard and MQTT Explorer to see if the data was successfully ingested and published.

✓ Click to Reveal Answer

You should see a new WIS2 data notification published for the test station `0-20000-0-64400`, indicating that the data was successfully ingested and published.

Station data publishing status		
WSI	Publish last 24 hrs	Fail last 24 hrs
0-20000-0-64400	3	0

If you use the wrong path, you will see an error message in the logs.

2.6.6 Uploading Data Using a Python Script

In this exercise, we will use the MinIO Python client to copy data into MinIO.

MinIO provides a Python client, which can be installed as follows:

```
pip3 install minio
```

On your student VM, the 'minio' package for Python will already be installed.

In the `exercise-materials/data-ingest-exercises` directory, you will find an example script `copy_file_to_incoming.py` that can be used to copy files into MinIO.

Try to run the script to copy the sample data file `synop_202501030900.txt` into the `wis2box-incoming` bucket in MinIO as follows:

```
cd ~/wis2box-data/data-ingest-exercises
python3 copy_file_to_incoming.py synop_202501030900.txt
```

Note

You will get an error as the script is not configured to access the MinIO endpoint on your wis2box yet.

The script needs to know the correct endpoint for accessing MinIO on your wis2box. If wis2box is running on your host, the MinIO endpoint is available at `http://YOUR-HOST:9000`. The script also needs to be updated with your storage password and the path in the MinIO bucket to store the data.

? Update the Script and Ingest the CSV Data

Edit the script `copy_file_to_incoming.py` to address the errors, using one of the following methods: - From the command line: use the `nano` or `vim` text editor to edit the script. - Using WinSCP: start a new connection using File Protocol `SCP` and the same credentials as your SSH client. Navigate into the directory `wis2box-data/data-ingest-exercises` and edit `copy_file_to_incoming.py` using the built-in text editor.

Ensure that you:

- Define the correct MinIO endpoint for your host.
- Provide the correct storage password for your MinIO instance.
- Provide the correct path in the MinIO bucket to store the data.

Re-run the script to ingest the sample data file `synop_202501030900.txt` into MinIO:

```
python3 ~/wis2box-data/ ~/wis2box-data/synop_202501030900.txt
```

Ensure the errors are resolved.

Once you manage to run the script successfully, you will see a message indicating that the file was copied to MinIO, and you should see data notifications published by your wis2box instance in MQTT Explorer.

You can also check the Grafana dashboard to see if the data was successfully ingested and published.

Now that the script is working, you can try to copy other files into MinIO using the same script.

? Ingesting Binary Data in BUFR Format

Run the following command to copy the binary data file `bufr-example.bin` into the `wis2box-incoming` bucket in MinIO:

```
python3 copy_file_to_incoming.py bufr-example.bin
```

Check the Grafana dashboard and MQTT Explorer to see if the test data was successfully ingested and published. If you see any errors, try to resolve them.

? Verify the Data Ingest

How many messages were published to the MQTT broker for this data sample?

✓ Click to Reveal Answer

You will see errors reported in Grafana as the stations in the BUFR file are not defined in the station list of your wis2box instance.

If all stations used in the BUFR file are defined in your wis2box instance, you should see 10 messages published to the MQTT broker. Each notification corresponds to data for one station for one observation timestamp.

The plugin `wis2box.data.bufr4.ObservationDataBUFR` splits the BUFR file into individual BUFR messages and publishes one message for each station and observation timestamp.

2.6.7 Conclusion

✓ Congratulations!

In this practical session, you learned how to:

- Trigger the wis2box workflow by uploading data to MinIO using various methods.
- Debug common errors in the data ingestion process using the Grafana dashboard and the logs of your wis2box instance.
- Monitor WIS2 data notifications published by your wis2box in the Grafana dashboard and MQTT Explorer.

2.7 Data Conversion Tools



Learning Outcomes

By the end of this practical session, you will be able to:

- Access ecCodes command-line tools within the wis2box-api container
- Use the synop2bufr tool to convert FM-12 SYNOP reports to BUFR from the command line
- Trigger synop2bufr conversion via the wis2box-webapp
- Use the csv2bufr tool to convert CSV data to BUFR from the command line

2.7.1 Introduction

Data published on WIS2 should meet the requirements and standards defined by the various Earth system discipline / domain expert communities. To lower the barrier to data publication for land-based surface observations, wis2box provides tools to convert data to BUFR format. These tools are available via the wis2box-api container and can be used from the command line to test the data conversion process.

The main conversions currently supported by wis2box are FM-12 SYNOP reports to BUFR and CSV data to BUFR. FM-12 data is supported as it is still widely used and exchanged in the WMO community, while CSV data is supported to allow the mapping of data produced by automatic weather stations to BUFR format.

About FM-12 SYNOP

Surface weather reports from land surface stations have historically been reported hourly or at the main (00, 06, 12, and 18 UTC) and intermediate (03, 09, 15, 21 UTC) synoptic hours. Prior to the migration to BUFR, these reports were encoded in the plain text FM-12 SYNOP code form. While the migration to BUFR was scheduled to be complete by 2012, a large number of reports are still exchanged in the legacy FM-12 SYNOP format. Further information on the FM-12 SYNOP format can be found in the WMO Manual on Codes, Volume I.1 (WMO-No. 306, Volume I.1).

About ecCodes

The ecCodes library is a set of software libraries and utilities designed to decode and encode meteorological data in the GRIB and BUFR formats. It is developed by the European Centre for Medium-Range Weather Forecasts (ECMWF), see the [ecCodes documentation](#) for more information.

The wis2box software includes the ecCodes library in the base image of the wis2box-api container. This allows users to access the command-line tools and libraries from within the container. The ecCodes library is used within the wis2box-stack to decode and encode BUFR messages.

About csv2bufr and synop2bufr

In addition to ecCodes, the wis2box uses the following Python modules that work with ecCodes to convert data to BUFR format:

- **synop2bufr**: to support the legacy FM-12 SYNOP format traditionally used by manual observers. The synop2bufr module relies on additional station metadata to encode additional parameters in the BUFR file. See the [synop2bufr repository at GitHub](#)
- **csv2bufr**: to enable the conversion of CSV-extract produced by automated weather stations to BUFR format. The csv2bufr module is used to convert CSV data to BUFR format using a mapping template that defines how the CSV data should be mapped to the BUFR format. See the [csv2bufr repository at GitHub](#)

These modules can be used standalone or as part of the wis2box stack.

2.7.2 Preparation

Prerequisites

- Ensure that your wis2box has been configured and started
- Ensure you have set up a dataset and configured at least one station in your wis2box
- Connect to the MQTT broker of your wis2box instance using MQTT Explorer
- Open the wis2box web application (`http://YOUR-HOST/wis2box-webapp`) and ensure you are logged in
- Open the Grafana dashboard for your instance by going to `http://YOUR-HOST:3000`

To use the BUFR command-line tools, you will need to be logged in to the wis2box-api container. Unless specified otherwise, all commands should be run on this container. You will also need to have MQTT Explorer open and connected to your broker.

First, connect to your student VM via your SSH client and copy the exercise materials to the wis2box-api container:

```
docker cp ~/exercise-materials/data-conversion-exercises wis2box-api:/root
```

Then log in to the wis2box-api container and change to the directory where the exercise materials are located:

```
cd ~/wis2box
python3 wis2box-ctl.py login wis2box-api
cd /root/data-conversion-exercises
```

Confirm that the tools are available, starting with ecCodes:

```
bufr_dump -V
```

You should get the following response:

```
ecCodes Version 2.36.0
```

Next, check the synop2bufr version:

```
synop2bufr --version
```

You should get the following response:

```
synop2bufr, version 0.7.0
```

Next, check csv2bufr:

```
csv2bufr --version
```

You should get the following response:

```
csv2bufr, version 0.8.5
```

2.7.3 ecCodes command-line tools

The ecCodes library included in the wis2box-api container provides a number of command-line tools for working with BUFR files. The next exercises demonstrate how to use `bufr_ls` and `bufr_dump` to check the content of a BUFR file.

bufr_ls

In this first exercise, you will use the `bufr_ls` command to inspect the headers of a BUFR file and determine the type of the contents of the file.

Use the following command to run `bufr_ls` on the file `bufr-cli-ex1.bufr4`:

```
bufr_ls bufr-cli-ex1.bufr4
```

You should see the following output:

```
bufr-cli-ex1.bufr4
centre                masterTablesVersionNumber  localTablesVersionNumber  typicalDate
typicalTime           numberOfSubsets
cnmc                   29                          0                          20231002
000000                1
1 of 1 messages in bufr-cli-ex1.bufr4

1 of 1 total messages in 1 file
```

Various options can be passed to `bufr_ls` to change both the format and header fields printed.

 Question

What would be the command to list the previous output in JSON format?

You can run the command `bufr_ls` with the `-h` flag to see the available options.

 Click to reveal answer

You can change the output format to JSON using the `-j` flag, i.e.

```
bufr_ls -j bufr-cli-ex1.bufr4
```

When run, this should give you the following output:

```
{ "messages" : [  
  {  
    "centre": "cnmc",  
    "masterTablesVersionNumber": 29,  
    "localTablesVersionNumber": 0,  
    "typicalDate": 20231002,  
    "typicalTime": "000000",  
    "numberOfSubsets": 1  
  }  
]}
```

The output printed represents the values of some of the header keys in the BUFR file.

On its own, this information is not very informative, with only limited information on the file contents provided.

When examining a BUFR file, we often want to determine the type of data contained in the file and the typical date/time of the data in the file. This information can be listed using the `-p` flag to select the headers to output. Multiple headers can be included using a comma-separated list.

You can use the following command to list the data category, sub-category, typical date, and time:

```
bufr_ls -p dataCategory,internationalDataSubCategory,typicalDate,typicalTime -j bufr-cli-ex1.bufr4
```

 Question

Execute the previous command and interpret the output using [Common Code Table C-13](#) to determine the data category and sub-category.

What type of data (data category and sub-category) is contained in the file? What is the typical date and time for the data?

 Click to reveal answer 

```
{ "messages" : [  
  {  
    "dataCategory": 2,  
    "internationalDataSubCategory": 4,  
    "typicalDate": 20231002,  
    "typicalTime": "000000"  
  }  
]}
```

From this, we see that:

- The data category is 2, indicating **"Vertical soundings (other than satellite)"** data.
- The international sub-category is 4, indicating **"Upper-level temperature/humidity/wind reports from fixed-land stations (TEMP)"** data.
- The typical date and time are 2023-10-02 and 00:00:00z, respectively.

bufr_dump

The `bufr_dump` command can be used to list and examine the contents of a BUFR file, including the data itself.

Try to run the `bufr_dump` command on the second example file `bufr-cli-ex2.bufr4`:

```
bufr_dump bufr-cli-ex2.bufr4
```

This results a JSON that can be hard to parse, try using the `-p` flag to output the data in plain text (key=value format):

```
bufr_dump -p bufr-cli-ex2.bufr4
```

You should will see a large number of keys as output, many of which are missing. This is typical with real world data as not all the eccodes keys are populated with reported data.

You can use the `grep` command to filter the output and only show the keys that are not missing. For example, to show all the keys that are not missing, you can use the following command:

```
bufr_dump -p bufr-cli-ex2.bufr4 | grep -v MISSING
```

Question

What is the pressured reduced to mean sea level reported in the BUFR file `bufr-cli-ex2.bufr4` ?

 Click to reveal answer

Using the following command:

```
bufr_dump -p bufr-cli-ex2.bufr4 | grep -i 'pressureReducedToMeanSeaLevel'
```

You should see the following output:

```
pressureReducedToMeanSeaLevel=105590
```

This indicates that the pressure reduced to mean sea level is 105590 Pa (1055.90 hPa).

 Question

What is the WIGOS station identifier of the station that reported the data in the BUFR file `bufr-cli-ex2.bufr4` ?

 Click to reveal answer

Using the following command:

```
bufr_dump -p bufr-cli-ex2.bufr4 | grep -i 'wigos'
```

You should see the following output:

```
wigosIdentifierSeries=0  
wigosIssuerOfIdentifier=20000  
wigosIssueNumber=0  
wigosLocalIdentifierCharacter="99100"
```

This indicates that the WIGOS station identifier is `0-20000-0-99100`.

2.7.4 synop2bufr conversion

Next, let's look at how to convert FM-12 SYNOP data to BUFR format using the `synop2bufr` module. The `synop2bufr` module is used to convert FM-12 SYNOP data to BUFR format. The module is installed in the `wis2box-api` container and can be used from the command line as follows:

```
synop2bufr data transform \  
  --metadata <station-metadata.csv> \  
  --output-dir <output-directory-path> \  
  --year <year-of-observation> \  
  --month <month-of-observation> \  
  <input-fm12.txt>
```

The `--metadata` argument is used to specify the station metadata file, that provide additional information to be encoded in the BUFR file. The `--output-dir` argument is used to specify the directory where the converted BUFR files will be written. The `--year` and `--month` arguments are used to specify the year and month of the observation.

The `synop2bufr` module is also used in the `wis2box-webapp` to convert FM-12 SYNOP data to BUFR format using a web-based input form.

The next few exercises will demonstrate how the `synop2bufr` -module works and how to use it to convert FM-12 SYNOP data to BUFR format.

review the example SYNOP message

Inspect the example SYNOP message file for this exercise `synop_message.txt` :

```
cd /root/data-conversion-exercises
more synop_message.txt
```

? Question

How many SYNOP reports are in this file?

✓ Click to reveal answer

The output shows the following:

```
AAXX 21121
15015 02999 02501 10103 21090 39765 42952 57020 60001=
15020 02997 23104 10130 21075 30177 40377 58020 60001 81041=
15090 02997 53102 10139 21075 30271 40364 58031 60001 82046=
```

There are 3 SYNOP reports in the file, corresponding to 3 different stations (identified by the 5-digit traditional station identifiers: 15015, 15020, and 15090). Note that the end of each report is marked by the `=` character.

review the station list

The `--metadata` argument requires a CSV file using a pre-defined format, a working example is provided in the file `station_list.csv` :

Use the following command to inspect the contents of the `station_list.csv` file:

```
more station_list.csv
```

 Question

How many stations are listed in the station list? What are the WIGOS station identifiers of the stations?

 Click to reveal answer

The output shows the following:

```
station_name,wigos_station_identifiser,traditional_station_identifiser,facility_type,latitude,longitu
de,elevation,barometer_height,territory_name,wmo_region
OCNA SUGATAG,0-20000-0-15015,15015,landFixed,47.7770616258,23.9404602638,503.0,504.0,ROU,europe
BOTOSANI,0-20000-0-15020,15020,landFixed,47.7356532437,26.6455501701,161.0,162.1,ROU,europe
```

This corresponds to the station metadata for 2 stations: for the WIGOS station identifiers `0-20000-0-15015` and `0-20000-0-15020`.

convert SYNOP to BUFR

Next, use the following command to convert the FM-12 SYNOP message to BUFR format:

```
synop2bufr data transform --metadata station_list.csv --output-dir ./ --year 2024 --month 09
synop_message.txt
```

 Question

How many BUFR files were created ? What does the WARNING message in the output mean?

 Click to reveal answer

The output shows the following:

```
[WARNING] Station 15090 not found in station file
```

If you check the content of your directory with the `ls -lh`, you should see 2 new BUFR files were created: `WIGOS_0-20000-0-15015_20240921T120000.bufr4` and `WIGOS_0-20000-0-15020_20240921T120000.bufr4`.

The warning message indicates that the station with the traditional station identifier `15090` was not found in the station list file `station_list.csv`. This means that the SYNOP report for this station was not converted to BUFR format.

 Question

Check the content of the BUFR file `WIGOS_0-20000-0-15015_20240921T120000.bufr4` using the `bufr_dump` command.

Can you verify the information provided in the `station_list.csv` file is present in the BUFR file?

 Click to reveal answer

You can use the following command to check the content of the BUFR file:

```
bufr_dump -p WIGOS_0-20000-0-15015_20240921T120000.bufr4 | grep -v MISSING
```

You will note the following output:

```
wigosIdentifierSeries=0
wigosIssuerOfIdentifier=20000
wigosIssueNumber=0
wigosLocalIdentifierCharacter="15015"
blockNumber=15
stationNumber=15
stationOrSiteName="OCNA SUGATAG"
stationType=1
year=2024
month=9
day=21
hour=12
minute=0
latitude=47.7771
longitude=23.9405
heightOfStationGroundAboveMeanSeaLevel=503
heightOfBarometerAboveMeanSeaLevel=504
```

Note that this includes the data provided by the `station_list.csv` file.

SYNOP Form in wis2box-webapp

The `synop2bufr` module is also used in the wis2box-webapp to convert FM-12 SYNOP data to BUFR format using a web-based input form. To test this, go to the `http://YOUR-HOST/wis2box-webapp` and log in.

Select the `SYNOP Form` from the menu on the left and copy paste the contents of the `synop_message.txt` file:

```
AAXX 21121
15015 02999 02501 10103 21090 39765 42952 57020 60001=
15020 02997 23104 10130 21075 30177 40377 58020 60001 81041=
15090 02997 53102 10139 21075 30271 40364 58031 60001 82046=
```

Into the `SYNOP message` text area:

Submit FM 12–XIV Ext. SYNOP Bulletin

See the WMO [Manual on Codes, Volume I.1](#), for a description of the FM 12-XIV Ext. SYNOP format

04/2025 ×

Month and year in
UTC

FM 12

AAXX 21121

15015 02999 02501 10103 21090 39765 42952 57020 60001=

15020 02997 23104 10130 21075 30177 40377 58020 60001 81041=

15090 02997 53102 10139 21075 30271 40364 58031 60001 82046=

Raw FM 12 bulletin

Dataset Identifier

urn:wmo:md:my-center-id:synop-test-dataset

this is my test-data sets for publishing data on weather/surface-based-observation/synop

wis2box auth token for 'processes/wis2box'

....



SUBMIT



Publish on
WIS2

? Question

Are you able to submit the form? What is the result?

 Click to reveal answer

You need to select a dataset and provide the token for "processes/wis2box" you created in the previous exercise to submit the form.

If you provide an invalid token, you will see:

- Result: Unauthorized, please provide a valid 'processes/wis2box' token

If you provide a valid token, you will see "WARNINGS: 3". Click on the "WARNINGS" to open the dropdown which will show:

- Station 15015 not found in station file
- Station 15020 not found in station file
- Station 15090 not found in station file

To convert this data to BUFR format you would need to configure the corresponding stations in your wis2box and ensure that the stations are associated to the topic for your dataset.

 Note

In the exercise for [ingesting-data-for-publication](#) you ingested the file "synop_202412030900.txt" and it was converted to BUFR format by the synop2bufr module.

In the automated workflow in the wis2box, the year and month are automatically extracted from the filename and used to populate the `--year` and `--month` arguments required by synop2bufr, while the station metadata is automatically extracted from the station configuration in the wis2box.

2.7.5 csv2bufr conversion

 Note

Make sure you are still logged in the wis2box-api container and in the directory `/root/data-conversion-exercises`, if you exited the container in the previous exercise, you can log in again as follows:

```
cd ~/wis2box
python3 wis2box-ctl.py login wis2box-api
cd /root/data-conversion-exercises
```

Now let's look at how to convert CSV data to BUFR format using the `csv2bufr` module. The module is installed in the wis2box-api container and can be used from the command line as follows:

```
csv2bufr data transform \
  --bufr-template <bufr-mapping-template> \
  <input-csv-file>
```

The `--bufr-template` argument is used to specify the BUFR mapping template file, which provides mapping between the input CSV data and the output BUFR data is specified in a JSON file. Default mapping templates are installed in the directory `/opt/csv2bufr/templates` in the `wis2box-api` container.

review the example CSV file

Review the content of the example CSV file `aws-example.csv`:

```
more aws-example.csv
```

? Question

How many rows of data are in the CSV file? What is the WIGOS station identifier of the stations reporting in the CSV file?

? Click to reveal answer



The output shows the following:

```
wsi_series,wsi_issuer,wsi_issue_number,wsi_local,wmo_block_number,wmo_station_number,station_type,y
ear,month,day,hour,minute,latitude,longitude,station_height_above_msl,barometer_height_above_msl,st
ation_pressure,msl_pressure,geopotential_height,thermometer_height,air_temperature,dewpoint_tempera
ture,relative_humidity,method_of_ground_state_measurement,ground_state,method_of_snow_depth_measure
ment,snow_depth,precipitation_intensity,anemometer_height,time_period_of_wind,wind_direction,wind_s
peed,maximum_wind_gust_direction_10_minutes,maximum_wind_gust_speed_10_minutes,maximum_wind_gust_di
rection_1_hour,maximum_wind_gust_speed_1_hour,maximum_wind_gust_direction_3_hours,maximum_wind_gust
_speed_3_hours,rain_sensor_height,total_precipitation_1_hour,total_precipitation_3_hours,total_prec
ipitation_6_hours,total_precipitation_12_hours,total_precipitation_24_hours
0,20000,0,60355,60,355,1,2024,3,31,1,0,47.77706163,23.94046026,503,504.43,100940,101040,1448,5,298.
15,294.55,80,3,1,1,0,0.004,10,-10,30,3,30,5,40,9,20,11,2,4.7,5.3,7.9,9.5,11.4
0,20000,0,60355,60,355,1,2024,3,31,2,0,47.77706163,23.94046026,503,504.43,100940,101040,1448,5,25.
294.55,80,3,1,1,0,0.004,10,-10,30,3,30,5,40,9,20,11,2,4.7,5.3,7.9,9.5,11.4
0,20000,0,60355,60,355,1,2024,3,31,3,0,47.77706163,23.94046026,503,504.43,100940,101040,1448,5,298.
15,294.55,80,3,1,1,0,0.004,10,-10,30,3,30,5,40,9,20,11,2,4.7,5.3,7.9,9.5,11.4
```

The first row of the CSV file contains the column headers, which are used to identify the data in each column.

After the header row, there are 3 rows of data, representing 3 weather observation from the same station with WIGOS station identifier `0-20000-0-60355` at three different timestamps `2024-03-31 01:00:00`, `2024-03-31 02:00:00`, and `2024-03-31 03:00:00`.

review the aws-template

The wis2box-api includes a set of predefined BUFR mapping templates that are installed in the directory `/opt/csv2bufr/templates`.

Check the content of the directory `/opt/csv2bufr/templates`:

```
ls /opt/csv2bufr/templates
```

You should see the following output:

```
CampbellAfrica-v1-template.json  aws-template.json  daycli-template.json
```

Let's check the content of the `aws-template.json` file:

```
cat /opt/csv2bufr/templates/aws-template.json
```

This returns a large JSON file, providing the mapping for 43 CSV columns.

? Question

What CSV column is mapped to the eccodes key `airTemperature`? What are the valid minimum and maximum values for this key?

✓ Click to reveal answer

Using the following command to filter the output:

```
cat /opt/csv2bufr/templates/aws-template.json | grep -i airTemperature
```

You should see the following output:

```
{"eccodes_key": "#1#airTemperature", "value": "data:air_temperature", "valid_min": "const:193.15",  
"valid_max": "const:333.15"},
```

The value that will be encoded for the eccodes key `airTemperature` will be taken from the data in the CSV column: **air_temperature**.

The minimum and maximum values for this key are `193.15` and `333.15`, respectively.

? Question

What CSV column is mapped to the eccodes key `internationalDataSubCategory`? What is the value of this key?

✓ Click to reveal answer



Using the following command to filter the output:

```
cat /opt/csv2bufr/templates/aws-template.json | grep -i internationalDataSubCategory
```

You should see the following output:

```
{"eccodes_key": "internationalDataSubCategory", "value": "const:2"},
```

There is no CSV column mapped to the eccodes key `internationalDataSubCategory`, instead the constant value 2 is used and will be encoded in all BUFR files produced with this mapping template.

convert CSV to BUFR

Let's attempt to convert the file to BUFR format using the `csv2bufr` command:

```
csv2bufr data transform --bufr-template aws-template ./aws-example.csv
```

? Question

How many BUFR files were created?

✓ Click to reveal answer



The output shows the following:

```
CLI:    ... Transforming ./aws-example.csv to BUFR ...
CLI:    ... Processing subsets:
CLI:    .... 384 bytes written to ./WIGOS_0-20000-0-60355_20240331T010000.bufr4
#1#airTemperature: Value (25.0) out of valid range (193.15 - 333.15).; Element set to missing
CLI:    .... 384 bytes written to ./WIGOS_0-20000-0-60355_20240331T020000.bufr4
CLI:    .... 384 bytes written to ./WIGOS_0-20000-0-60355_20240331T030000.bufr4
CLI:    End of processing, exiting.
```

The output indicates that 3 BUFR files were created: `WIGOS_0-20000-0-60355_20240331T010000.bufr4`, `WIGOS_0-20000-0-60355_20240331T020000.bufr4`, and `WIGOS_0-20000-0-60355_20240331T030000.bufr4`.

To check the content of the BUFR files while ignoring the missing values, you can use the following command:

```
bufr_dump -p WIGOS_0-20000-0-60355_20240331T010000.bufr4 | grep -v MISSING
```

 Question

What is the value of the eccodes key `airTemperature` in the BUFR file `WIGOS_0-20000-0-60355_20240331T010000.bufr4` ? How about in the BUFR file `WIGOS_0-20000-0-60355_20240331T020000.bufr4` ?

 Click to reveal answer

To filter the output, you can use the following command:

```
bufr_dump -p WIGOS_0-20000-0-60355_20240331T010000.bufr4 | grep -v MISSING | grep airTemperature
```

You should see the following output:

```
#1#airTemperature=298.15
```

While for the second file:

```
bufr_dump -p WIGOS_0-20000-0-60355_20240331T020000.bufr4 | grep -v MISSING | grep airTemperature
```

You get no result, indicating that the value for the key `airTemperature` is missing in the BUFR file `WIGOS_0-20000-0-60355_20240331T020000.bufr4` . The `csv2bufr` refused to encode the value `25.0` from the CSV-data as it is outside the valid range of `193.15` and `333.15` as defined in the mapping template.

Note that converting CSV to BUFR using one of the predefined BUFR mapping template has limitations:

- the CSV file must be in the format defined in the mapping template, i.e. the CSV column names must match the names defined in the mapping template
- you can only encode the keys defined in the mapping template
- the quality control checks are limited to the checks defined in the mapping template

For information on how to create and use custom BUFR-mapping-templates see the next practical exercise [csv2bufr-templates](#).

2.7.6 Conclusion

**Congratulations!**

In this practical session you have learned:

- how to access ecCodes command-line tools within the wis2box-api container
- how to use `synop2bufr` to convert FM-12 SYNOP reports to BUFR from the command line
- how to use the SYNOP Form in the wis2box-webapp to convert FM-12 SYNOP reports to BUFR
- how to use `csv2bufr` to convert CSV data to BUFR from the command line

2.8 CSV-to-BUFR mapping templates



Learning outcomes

By the end of this practical session, you will be able to:

- create a new BUFR mapping template for your CSV data
- edit and debug your custom BUFR mapping template from the command line
- configure the CSV-to-BUFR data plugin to use a custom BUFR mapping template
- use the built-in AWS and DAYCLI templates for converting CSV data to BUFR

2.8.1 Introduction

Comma-separated values (CSV) data files are often used for recording observational and other data in a tabular format. Most data loggers used to record sensor output are able to export the observations in delimited files, including in CSV. Similarly, when data are ingested into a database it is easy to export the required data in CSV formatted files.

The `wis2box csv2bufr` module provides a command line tool to convert CSV data to BUFR format. When using `csv2bufr` you need to provide a BUFR mapping template that maps CSV-columns to corresponding BUFR elements. If you don't want to create your own mapping template, you can use the built-in AWS and DAYCLI templates for converting CSV data to BUFR, but you will need to ensure that the CSV data you are using is in the correct format for these templates. If you want to decode parameters that are not included in the AWS and DAYCLI templates, you will need to create your own mapping template.

In this session you will learn how to create your own mapping template for converting CSV data to BUFR. You will also learn how to use the built-in AWS and DAYCLI templates for converting CSV data to BUFR.

2.8.2 Preparation

Make sure the `wis2box-stack` has been started with `python3 wis2box.py start`

Make sure that you have a web browser open with the MinIO UI for your instance by going to `http://YOUR-HOST:9000`. If you don't remember your MinIO credentials, you can find them in the `wis2box.env` file in the `wis2box` directory on your student VM.

Make sure that you have MQTT Explorer open and connected to your broker using the credentials `everyone/everyone`.

2.8.3 Creating a mapping template

The csv2bufr module comes with a command line tool to create your own mapping template using a set of BUFR sequences and/or BUFR element as input.

To find specific BUFR sequences and elements you can refer to the BUFR tables at <https://confluence.ecmwf.int/display/ECC/BUFR+tables>.

csv2bufr mappings command line tool

To access the csv2bufr command line tool, you need to login to the wis2box-api container:

```
cd ~/wis2box
python3 wis2box-ctl.py login wis2box-api
```

To to print the help page for the command `csv2bufr mapping`:

```
csv2bufr mappings --help
```

The help page show 2 subcommands:

- `csv2bufr mappings create` : Create a new mapping template
- `csv2bufr mappings list` : List the mapping templates available in the system



csv2bufr mapping list

The `csv2bufr mapping list` command will show you the mapping templates available in the system. Default templates are stored in the directory `/opt/wis2box/csv2bufr/templates` in the container.

To share custom mapping templates with the system you can store them in the directory defined by `$CSV2BUFR_TEMPLATES`, which is set to `/data/wis2box/mappings` by default in the container. Since the directory `/data/wis2box/mappings` in the container is mounted to the directory `$WIS2BOX_HOST_DATADIR/mappings` on the host, you will find your custom mapping templates in the directory `$WIS2BOX_HOST_DATADIR/mappings` on the host.

Let's try to create a new custom mapping template using the `csv2bufr mapping create` command using as input BUFR sequence 301150 plus BUFR element 012101.

```
csv2bufr mappings create 301150 012101 --output /data/wis2box/mappings/my_custom_template.json
```

You can check the content of the mapping template you just created using the `cat` command:

```
cat /data/wis2box/mappings/my_custom_template.json
```

? Inspection of the mapping template

How many CSV columns are being mapped to BUFR elements? What is the CSV-header for each BUFR element being mapped?

✓ Click to reveal answer

The mapping template you created maps **5** CSV columns to BUFR elements, namely the 4 BUFR elements in sequence 301150 plus the BUFR element 012101.

The following CSV columns are being mapped to BUFR elements:

- **wigosIdentifierSeries** maps to "eccodes_key": "#1#wigosIdentifierSeries" (BUFR element 001125)
- **wigosIssuerOfIdentifier** maps to "eccodes_key": "#1#wigosIssuerOfIdentifier" (BUFR element 001126)
- **wigosIssueNumber** maps to "eccodes_key": "#1#wigosIssueNumber" (BUFR element 001127)
- **wigosLocalIdentifierCharacter** maps to "eccodes_key": "#1#wigosLocalIdentifierCharacter" (BUFR element 001128)
- **airTemperature** maps to "eccodes_key": "#1#airTemperature" (BUFR element 012101)

The mapping template you created misses important metadata about the observation that was made, the date and time of the observation, and the latitude and longitude of the station.

Next we will update the mapping template and add the following sequences :

- **301011** for Date (Year, month, day)
- **301012** for Time (Hour, minute)
- **301023** for Location (Latitude/longitude (coarse accuracy))

And the following elements :

- **010004** for Pressure
- **007031** for Barometer height above mean sea level

Execute the following command to update the mapping template:

```
csv2bufr mappings create 301150 301011 301012 301023 007031 012101 010004 --output /data/wis2box/mappings/my_custom_template.json
```

And inspect the content of the mapping template again:

```
cat /data/wis2box/mappings/my_custom_template.json
```

? Inspection of the updated mapping template

How many CSV columns are now being mapped to BUFR elements? What is the CSV-header for each BUFR element being mapped?

✓ Click to reveal answer

The mapping template you created now maps **18** CSV columns to BUFR elements: - 4 BUFR elements from BUFR sequence 301150 - 3 BUFR elements from BUFR sequence 301011 - 2 BUFR elements from BUFR sequence 301012 - 2 BUFR elements from BUFR sequence 301023 - BUFR element 007031 - BUFR element 012101

The following CSV columns are being mapped to BUFR elements:

- **wigosIdentifierSeries** maps to "eccodes_key": "#1#wigosIdentifierSeries" (BUFR element 001125)
- **wigosIssuerOfIdentifier** maps to "eccodes_key": "#1#wigosIssuerOfIdentifier" (BUFR element 001126)
- **wigosIssueNumber** maps to "eccodes_key": "#1#wigosIssueNumber" (BUFR element 001127)
- **wigosLocalIdentifierCharacter** maps to "eccodes_key": "#1#wigosLocalIdentifierCharacter" (BUFR element 001128)
- **year** maps to "eccodes_key": "#1#year" (BUFR element 004001)
- **month** maps to "eccodes_key": "#1#month" (BUFR element 004002)
- **day** maps to "eccodes_key": "#1#day" (BUFR element 004003)
- **hour** maps to "eccodes_key": "#1#hour" (BUFR element 004004)
- **minute** maps to "eccodes_key": "#1#minute" (BUFR element 004005)
- **latitude** maps to "eccodes_key": "#1#latitude" (BUFR element 005002)
- **longitude** maps to "eccodes_key": "#1#longitude" (BUFR element 006002)
- **heightOfBarometerAboveMeanSeaLevel** maps to "eccodes_key": "#1#heightOfBarometerAboveMeanSeaLevel" (BUFR element 007031)
- **airTemperature** maps to "eccodes_key": "#1#airTemperature" (BUFR element 012101)
- **nonCoordinatePressure** maps to "eccodes_key": "#1#nonCoordinatePressure" (BUFR element 010004)

Check the content of the file `custom_template_data.csv` in the directory `/root/data-conversion-exercises`:

```
cat /root/data-conversion-exercises/custom_template_data.csv
```

Note that the headers of this CSV file are the same as the CSV headers in the mapping template you created.

To test the data conversion we can use the `csv2bufr` command line tool to convert the CSV file to BUFR using the mapping template we created:

```
csv2bufr data transform --bufr-template my_custom_template /root/data-conversion-exercises/custom_template_data.csv
```

You should see the following output:

```
CLI:    ... Transforming /root/data-conversion-exercises/custom_template_data.csv to BUFR ...
CLI:    ... Processing subsets:
CLI:    ..... 94 bytes written to ./WIGOS_0-20000-0-15015_20250412T210000.bufr4
CLI:    End of processing, exiting.
```

? Check the content of the BUFR file

How can you check the content of the BUFR file you just created and verify that it has encoded the data correctly?

✓ Click to reveal answer

You can use the `bufr_dump -p` command to check the content of the BUFR file you just created. The command will show you the content of the BUFR file in a human readable format.

```
bufr_dump -p ./WIGOS_0-20000-0-15015_20250412T210000.bufr4
```

In the output you will see values for the BUFR elements you mapped in the template, for example the "airTemperature" will show:

```
airTemperature=298.15
```

You can now exit the container:

```
exit
```

Using the mapping template in the wis2box

To ensure the new mapping template is recognized by the wis2box-api container, you need to restart the container:

```
docker restart wis2box-api
```

You can now configure your dataset in the wis2box-webapp to use the custom mapping template for the CSV to BUFR conversion plugin.

The wis2box-webapp will automatically detect the mapping template you created and make it available in the list of templates for the CSV to BUFR conversion plugin.

Click on the dataset you created in the previous practical session and click on "UPDATE" next to the plugin with name "CSV data converted to BUFR":

Dataset Mappings Editor

Plugins in use	File extension	File pattern		
BUFR data converted to BUFR	bin	^.*\.bin\$	UPDATE	DELETE
FM-12 data converted to BUFR	txt	^.*_(\d{4})(\d{2}).*.txt\$	UPDATE	DELETE
BUFR data converted to BUFR	b	^.*\.b\$	UPDATE	DELETE
CSV data converted to BUFR	csv	^.*\.csv\$	UPDATE	DELETE
BUFR data converted to GeoJSON	bufr4	^WIGOS_(\d+\.d+\.d+\.w+)_.*\.bufr4\$	UPDATE	DELETE

[ADD A PLUGIN](#)

You should see the new template you created in the list of available templates:

Plugin Configuration

Plugin Name

CSV data converted to BUFR

File Extension

CSV

Template

my_custom_template

WIS2 Buckets

Incoming

my_custom_template

DayCLI

WIS2-pilot-template-2021

AWS

☒ Notify

SAVE

 Hint

Note that if you don't see the new template you created, try refreshing the page or opening it in a new incognito window.

For now keep the default selection of the AWS template (click on the top right to close the plugin configuration).

2.8.4 Using the 'AWS' template

The 'AWS' template provides mapping template for converting CSV data to BUFR sequence 301150, 307096, in support of minimum GBON requirements.

The description of the AWS template can be found here [aws-template](#).

Review the aws-example input data

Download the example for this exercise from the link below:

[aws-example.csv](#)

Open the file you downloaded in an editor and inspect the content:

Question

Examining the date, time and identify fields (WIGOS and traditional identifiers) what do you notice? How would today's date be represented?

Click to reveal answer

Each column contains a single piece of information. For example the date is split into year, month and day, mirroring how the data are stored in BUFR. Today's date would be split across the columns "year", "month" and "day". Similarly, the time needs to be split into "hour" and "minute" and the WIGOS station identifier into its respective components.

Question

Looking at the data file how are missing data encoded?

Click to reveal answer

Missing data within the file are represented by empty cells. In a CSV file this would be encoded by `,`. Note that this is an empty cell and not encoded as a zero length string, e.g. `,"",`.

Missing data

It is recognized that data may be missing for a variety of reasons, whether due to sensor failure or the parameter not being observed. In these cases missing data can be encoded as per the above answer, the other data in the report remain valid.

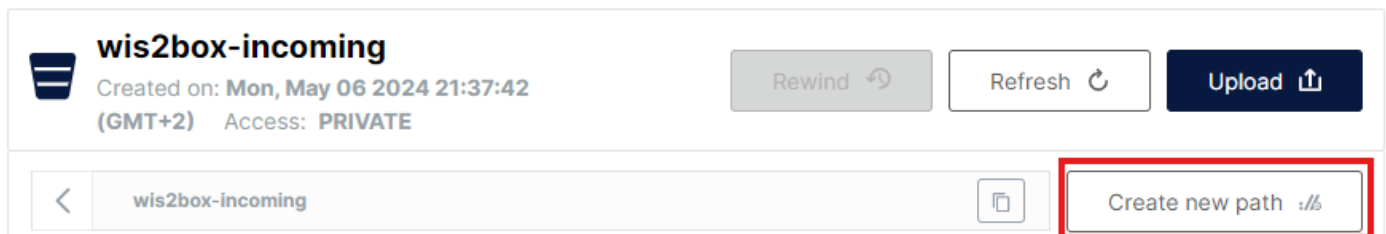
Update the example file

Update the example file you downloaded to use today's date and time and change the WIGOS station identifiers to use stations you have registered in the wis2box-webapp.

Upload the data to MinIO and check the result

Navigate to the MinIO UI and log in using the credentials from the `wis2box.env` file.

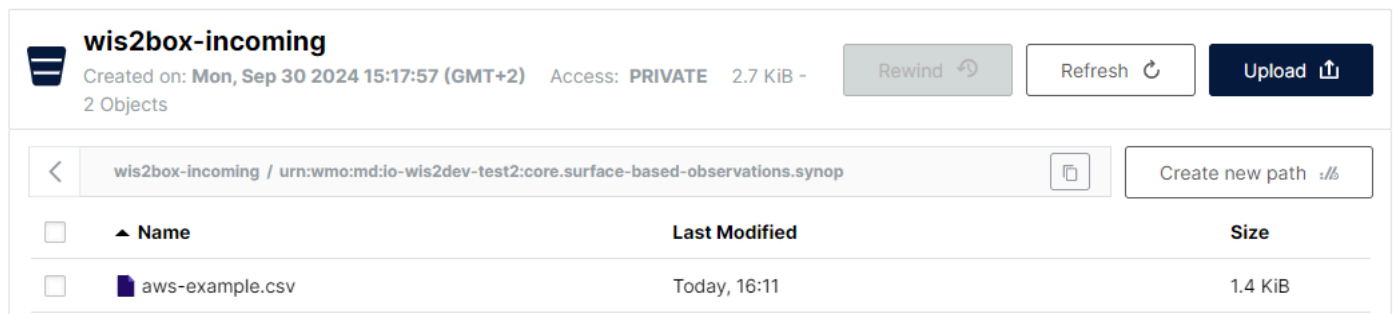
Navigate to the **wis2box-incoming** and click the button "Create new path":



Create a new folder in the MinIO bucket that matches the dataset-id for the dataset you created with the template='weather/surface-weather-observations/synop':



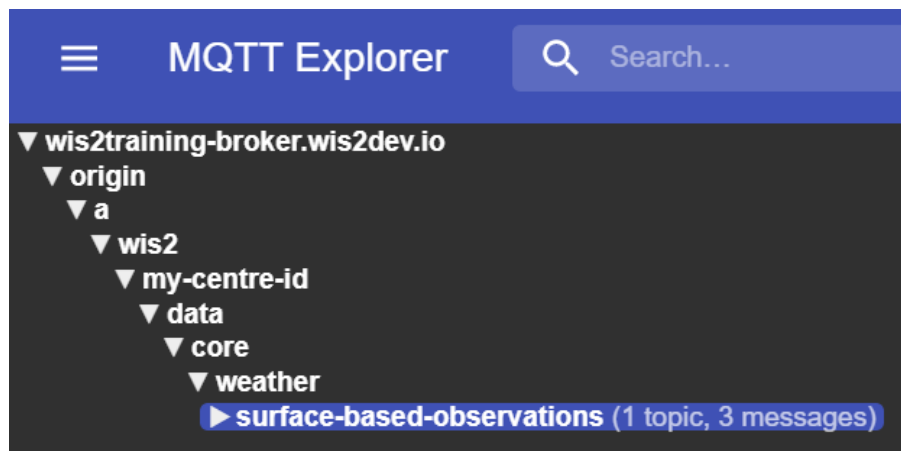
Upload the example file you downloaded to the folder you created in the MinIO bucket:



Check the Grafana dashboard at `http://YOUR-HOST:3000` to see if there are any WARNINGS or ERRORS. If you see any, try to fix them and repeat the exercise.

Check the MQTT Explorer to see if you receive WIS2 data-notifications.

If you successfully ingested the data you should see 3 notifications in MQTT explorer on the topic `origin/a/wis2/<centre-id>/data/weather/surface-weather-observations/synop` for the 3 stations you reported data for:



2.8.5 Using the 'DayCLI' template

The **DayCLI** template provides a mapping template for converting daily climate CSV data to BUFR sequence 307075, in support of reporting daily climate data.

The description of the DAYCLI template can be found here [daycli-template](#).

To share this data on WIS2 you will need to create a new dataset in the wis2box-webapp that has the correct WIS2 Topic Hierarchy and that uses the DAYCLI template for converting CSV data to BUFR.

Creating a wis2box dataset for publishing DAYCLI messages

Go to the dataset editor in the wis2box-webapp and create a new dataset. Use the same centre-id as in the previous practical sessions and select **Data Type='climate/surface-based-observations/daily'**:

Please enter some initial information

Centre ID

my-centre-id

Data Type

climate/surface-based-observations/daily

RETURN HOME 

CONTINUE TO FORM 

Click "CONTINUE TO FORM" and add a description for your dataset, set the bounding box and provide the contact information for the dataset. Once you are done filling out all the sections, click 'VALIDATE FORM' and check the form.

Review the data-plugins for the datasets. Click on "UPDATE" next to the plugin with name "CSV data converted to BUFR" and you will see the template is set to **DayCLI**:

Plugin Configuration

Plugin Name

CSV data converted to BUFR

File Extension

CSV

Template

DayCLI

WIS2 Buckets

Incoming

File Pattern (Regex)

^.*\.csv\$

 Notify

SAVE

Close the plugin configuration and submit the form using the authentication token you created in the previous practical session.

You should know have a second dataset in the wis2box-webapp that is configured to use the DAYCLI template for converting CSV data to BUFR.

Review the daycli-example input data

Download the example for this exercise from the link below:

[daycli-example.csv](#)

Open the file you downloaded in an editor and inspect the content:

Question

What additional variables are included in the daycli template?

Click to reveal answer

The daycli template includes important metadata on the instrument siting and measurement quality classifications for temperature and humidity, quality control flags and information on how the daily average temperature has been calculated.

Update the example file

The example file contains one row of data for each day in a month, and reports data for one station. Update the example file you downloaded to use today's date and time and change the WIGOS station identifiers to use a station you have registered in the wis2box-webapp.

Upload the data to MinIO and check the result

As before, you will need to upload the data to the 'wis2box-incoming' bucket in MinIO to be processed by the csv2bufr converter. This time you will need to create a new folder in the MinIO bucket that matches the dataset-id for the dataset you created with the template='climate/surface-based-observations/daily' which will be different from the dataset-id you used in the previous exercise:

wis2box-incoming
Created on: **Mon, Sep 30 2024 15:17:57 (GMT+2)** Access: **PRIVATE** 5.5 KiB - 1 Object

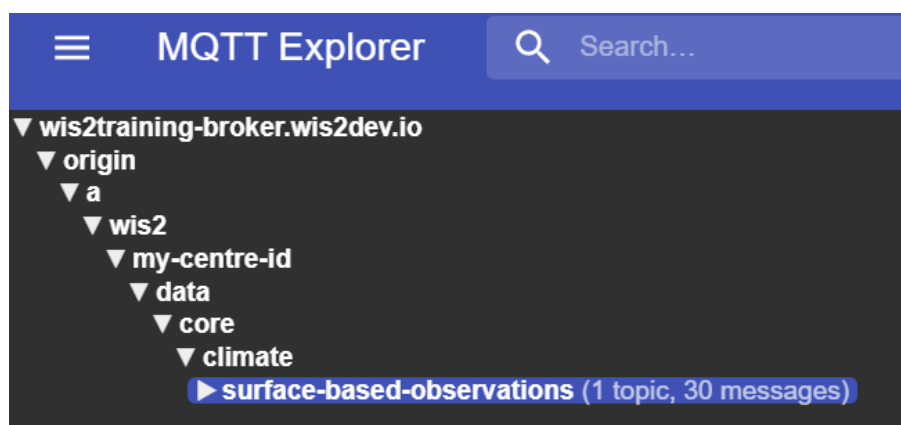
Rewind Refresh Upload

< wis2box-incoming / urn:wmo:md:my-centre-id:core.climate.surface.daily Create new path ://

☐	▲ Name	Last Modified	Size
☐	 daycli-example.csv	Today, 11:51	5.5 KiB

After uploading the data check there are no WARNINGS or ERRORS in the Grafana dashboard and check the MQTT Explorer to see if you receive WIS2 data-notifications.

If you successfully ingested the data you should see 30 notifications in MQTT explorer on the topic `origin/a/wis2/<centre-id>/data/climate/surface-based-observations/daily` for the 30 days in the month you reported data for:



2.8.6 Conclusion

✓ Congratulations

In this practical session you have learned:

- how to create a custom mapping template for converting CSV data to BUFR
- how to use the built-in AWS and DAYCLI templates for converting CSV data to BUFR

2.9 Adding GTS headers to WIS2 notifications

Learning outcomes

By the end of this practical session, you will be able to:

- configure a mapping between filename and GTS headers
- ingest data with a filename that matches the GTS headers
- view the GTS headers in the WIS2 notifications

2.9.1 Introduction

WMO Members wishing to stop their data transmission on GTS during the transition phase to WIS2 will need to add GTS headers to their WIS2 notifications. These headers enable the WIS2 to GTS gateway to forward the data to the GTS network.

This allows Members having migrated to using a WIS2 node for data publication to disable their MSS system and ensure that their data is still available to Members not yet migrated to WIS2.

The GTS property in the WIS2 Notification Message needs to be added as an additional property to the WIS2 Notification Message. The GTS property is a JSON object that contains the GTS headers that are required for the data to be forwarded to the GTS network.

```
{
  "gts": {
    "ttaaii": "FTAE31",
    "cccc": "VTBB"
  }
}
```

Within wis2box you can add this to WIS2 Notifications automatically by providing an additional file named `gts_headers_mapping.csv` that contains the required information to map the GTS headers to the incoming filenames.

This file should be placed in the directory defined by `WIS2BOX_HOST_DATADIR` in your `wis2box.env` and should have the following columns:

- `string_in_filepath`: a string that is part of the filename that will be used to match the GTS headers
- `TTAAii`: the TTAAii header to be added to the WIS2 notification
- `CCCC`: the CCCC header to be added to the WIS2 notification

2.9.2 Preparation

Ensure you have SSH access to your student VM and that your wis2box instance is up and running.

Make sure that you are connected to the MQTT broker of your wis2box instance using MQTT Explorer. You can use the public credentials `everyone/everyone` to connect to the broker.

Make sure you have a web browser open with the Grafana dashboard for your instance by going to `http://YOUR-HOST:3000`

2.9.3 creating `gts_headers_mapping.csv`

To add GTS headers to your WIS2 notifications, a CSV file is required that maps GTS headers to incoming filenames.

The CSV file should be named (exactly) `gts_headers_mapping.csv` and should be placed in the directory defined by `WIS2BOX_HOST_DATADIR` in your `wis2box.env`.

2.9.4 Providing a `gts_headers_mapping.csv` file

Copy the file `exercise-materials/gts-headers-exercises/gts_headers_mapping.csv` to your wis2box instance and place it in the directory defined by `WIS2BOX_HOST_DATADIR` in your `wis2box.env`.

```
cp ~/exercise-materials/gts-headers-exercises/gts_headers_mapping.csv ~/wis2box-data
```

Then restart the wis2box-management container to apply the changes:

```
docker restart wis2box-management
```

2.9.5 Ingesting data with GTS headers

Copy the file `exercise-materials/gts-headers-exercises/A_SMR001YRBK171200_C_EDZW_20240717120502.txt` to the directory defined by `WIS2BOX_HOST_DATADIR` in your `wis2box.env`:

```
cp ~/exercise-materials/gts-headers-exercises/A_SMR001YRBK171200_C_EDZW_20240717120502.txt  
~/wis2box-data
```

Then login to the **wis2box-management** container:

```
cd ~/wis2box  
python3 wis2box-ctl.py login
```

From the wis2box command line we can ingest the sample data file

`A_SMR001YRBK171200_C_EDZW_20240717120502.txt` into a specific dataset as follows:

```
wis2box data ingest -p /data/wis2box/A_SMR001YRBK171200_C_EDZW_20240717120502.txt --metadata-id  
urn:wmo:md:not-my-centre:core.surface-based-observations.synop
```

Make sure to replace the `metadata-id` option with the correct identifier for your dataset.

Check the Grafana dashboard to see if the data was ingested correctly. If you see any WARNINGS or ERRORS, try to fix them and repeat the exercise the `wis2box data ingest` command.

2.9.6 Viewing the GTS headers in the WIS2 Notification

Go to the MQTT Explorer and check for the WIS2 Notification Message for the data you just ingested.

The WIS2 Notification Message should contain the GTS headers you provided in the `gts_headers_mapping.csv` file.

2.9.7 Conclusion

Congratulations!

In this practical session, you learned how to: - add GTS headers to your WIS2 notifications - verify GTS headers are made available via your wis2box installation

2.10 Setting up a recommended dataset with access control



Learning outcomes

By the end of this practical session, you will be able to:

- create a new dataset with data policy 'recommended'
- add an access token to the dataset
- validate the dataset can not be accessed without the access token
- add the access token to HTTP headers to access the dataset

2.10.1 Introduction

Datasets that are not considered 'core' dataset in WMO can optionally be configured with an access control policy. wis2box provides a mechanism to add an access token to a dataset which will prevent users from downloading data unless they supply the access token in the HTTP headers.

2.10.2 Preparation

Ensure you have SSH access to your student VM and that your wis2box instance is up and running.

Make sure you are connected to the MQTT broker of your wis2box instance using MQTT Explorer. You can use the public credentials `everyone/everyone` to connect to the broker.

Ensure you have a web browser open with the wis2box-webapp for your instance by going to `http://YOUR-HOST/wis2box-webapp`.

2.10.3 Create a new dataset with data policy 'recommended'

Go to the 'dataset editor' page in the wis2box-webapp and create a new dataset. Select the Data Type = 'weather/surface-weather-observations/synop'.

Please enter some initial information

Centre ID	▼
Data Type	weather/surface-based-observations/synop ▼

[RETURN HOME !\[\]\(6605b201d6f14d9b3bcb8ab5f274d107_img.jpg\)](#)[CONTINUE TO FORM !\[\]\(96cc62f861fdd6e50510c0224a756dff_img.jpg\)](#)

For "Centre ID", use the same as you used in the previous practical sessions.

Click 'CONTINUE To FORM' to proceed.

In the dataset editor, set the data policy to 'recommended' (note that changing the data-policy will update the 'Topic Hierarchy'). Replace the auto-generated 'Local ID' with a descriptive name for the dataset, e.g. 'recommended-data-with-access-control':

Metadata Editor

Dataset Identification

Title Hourly synoptic observations from fixed-land stations (SYNOP) (int-wmo-example)	Description Recommended dataset to which access control will be applied	
Local ID recommended-data-with-access-control		
Identifier urn:wmo:md:int-wmo-example:recommended-data-with-access-control 		
Centre ID int-wmo-example	WMO Data Policy recommended ▼	Topic Hierarchy int-wmo-example/data/recommended/weather/surface-based-observations/synop

Continue to fill the required fields for Spatial Properties and Contact Information, and 'Validate form' to check for any errors.

Finally submit the dataset, using the previously create authentication token, and check that the new dataset is created in the wis2box-webapp.

Check MQTT-explorer to see that you receive the WIS2 Notification Message announcing the new Discovery Metadata record on the topic `origin/a/wis2/<your-centre-id>/metadata`.

2.10.4 Add an access token to the dataset

Login to the wis2box-management container,

```
cd ~/wis2box
python3 wis2box-ctl.py login
```

From command line inside the container you can secure a dataset using the `wis2box auth add-token` command, using the flag `--metadata-id` to specify the metadata-identifier of the dataset and the access token as an argument.

For example, to add the access token `S3cr3tT0k3n` to the dataset with metadata-identifier `urn:wmo:md:not-my-centre:core.surface-based-observations.synop`:

```
wis2box auth add-token --metadata-id urn:wmo:md:not-my-centre:reco.surface-based-
observations.synop S3cr3tT0k3n
```

Exit the wis2box-management container:

```
exit
```

2.10.5 Publish some data to the dataset

Copy the file `exercise-materials/access-control-exercises/aws-example.csv` to the directory defined by `WIS2BOX_HOST_DATADIR` in your `wis2box.env`:

```
cp ~/exercise-materials/access-control-exercises/aws-example.csv ~/wis2box-data
```

Then use WinSCP or a command line editor to edit the file `aws-example.csv` and update the WIGOS-station-identifiers in the input-data to match the stations you have in your wis2box instance.

Next, go to the station-editor in the wis2box-webapp. For each station you used in `aws-example.csv`, update the 'topic' field to match the 'topic' of the dataset you created in the previous exercise.

This station will now be associated to 2 topics, one for the 'core' dataset and one for the 'recommended' dataset:

Topics

origin/a/wis2/int-mlimper/data/core/weather/surface-based-observations/synop,
origin/a/wis2/int-mlimper/data/recommended/weather/surface-based-observations/synop

wis2box auth token for "collections/stations"

.....

SAVE

CANCEL

You will need to use your token for `collections/stations` to save the updated station data.

Next, login to the wis2box-management container:

```
cd ~/wis2box
python3 wis2box-ctl.py login
```

From the wis2box command line we can ingest the sample data file `aws-example.csv` into a specific dataset as follows:

```
wis2box data ingest -p /data/wis2box/aws-example.csv --metadata-id urn:wmo:md:not-my-
centre:reco.surface-based-observations.synop
```

Make sure to provide the correct metadata-identifier for your dataset and **check that you receive WIS2 data-notifications in MQTT Explorer**, on the topic `origin/a/wis2/<your-centre-id>/data/recommended/surface-based-observations/synop`.

Check the canonical link in the WIS2 Notification Message and copy/paste the link to the browser to try and download the data.

You should see a 403 Forbidden error.

2.10.6 Add the access token to HTTP headers to access the dataset

In order to demonstrate that the access token is required to access the dataset we will reproduce the error you saw in the browser using the command line function `wget`.

From the command line in your student VM, use the `wget` command with the canonical-link you copied from the WIS2 Notification Message.

```
wget <canonical-link>
```

You should see that the HTTP request returns with *401 Unauthorized* and the data is not downloaded.

Now add the access token to the HTTP headers to access the dataset.

```
wget --header="Authorization: Bearer S3cr3tT0k3n" <canonical-link>
```

Now the data should be downloaded successfully.

2.10.7 Conclusion

✓ Congratulations!

In this practical session, you learned how to:

- create a new dataset with data policy 'recommended'
- add an access token to the dataset
- validate the dataset can not be accessed without the access token
- add the access token to HTTP headers to access the dataset

2.11 Downloading and decoding data from WIS2

Learning outcomes!

By the end of this practical session, you will be able to:

- use the "wis2downloader" to subscribe to WIS2 data notifications and download data to your local system
- view the status of the downloads in the Grafana dashboard
- decode some downloaded data using the "decode-bufr-jupyter" container

2.11.1 Introduction

In this session you will learn how to setup a subscription to a WIS2 Broker and automatically download data to your local system using the "wis2downloader"-service included in the wis2box.

About wis2downloader

The wis2downloader is also available as a standalone service that can be run on a different system from the one that is publishing the WIS2 notifications. See [wis2downloader](#) for more information for using the wis2downloader as a standalone service.

If you like to develop your own service for subscribing to WIS2 notifications and downloading data, you can use the [wis2downloader source code](#) as a reference.

Other

The following tools can also be used to discover and access data from WIS2:

- [pywiscat](#) provides search capability atop the WIS2 Global Discovery Catalogue in support of reporting and analysis of the WIS2 Catalogue and its associated discovery metadata
- [pywis-pubsub](#) provides subscription and download capability of WMO data from WIS2 infrastructure services

2.11.2 Preparation

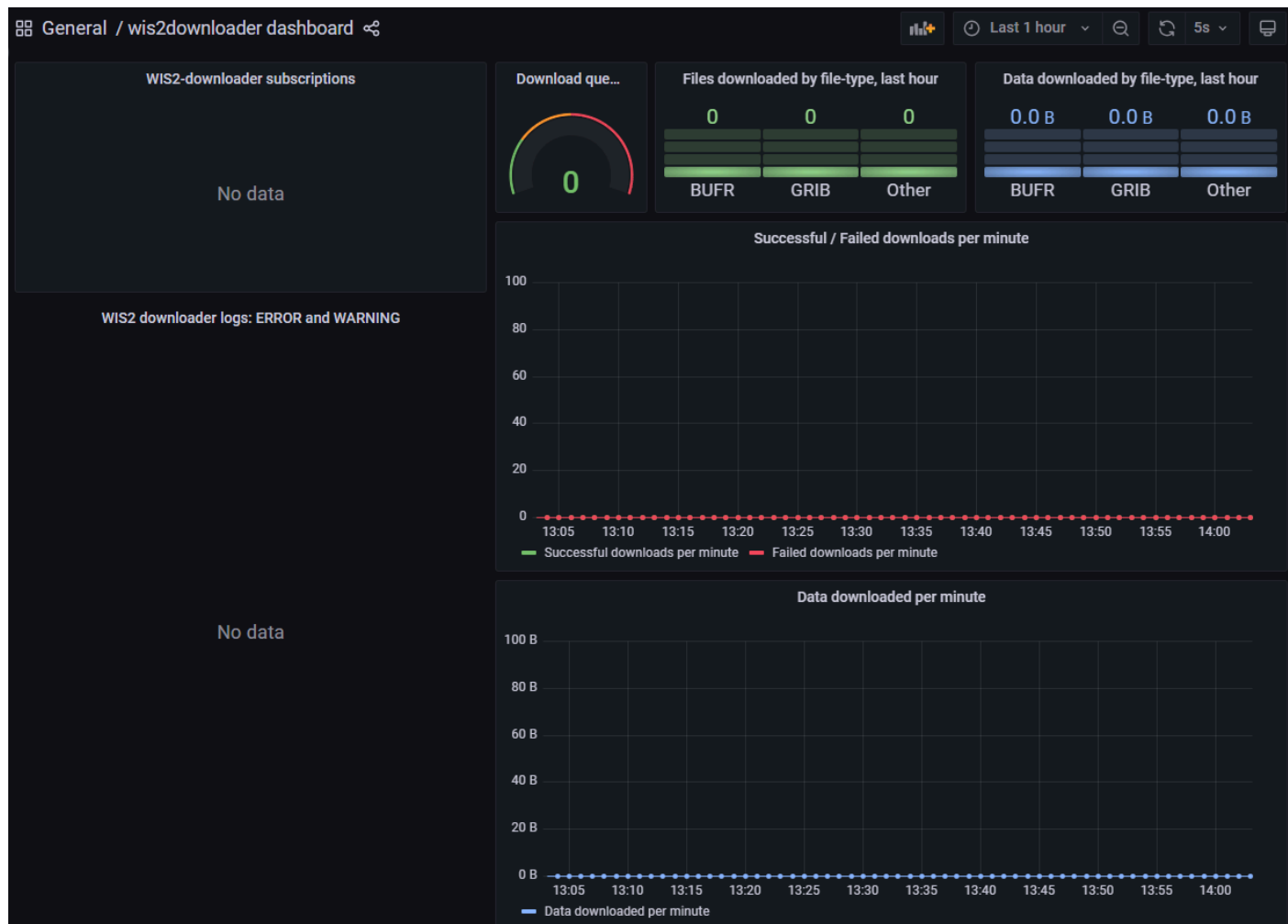
Before starting please login to your student VM and ensure your wis2box instance is up and running.

2.11.3 Viewing the wis2downloader dashboard in Grafana

Open a web browser and navigate to the Grafana dashboard for your wis2box instance by going to `http://YOUR-HOST:3000`.

Click on dashboards in the left-hand menu, and then select the **wis2downloader dashboard**.

You should see the following dashboard:



This dashboard is based on metrics published by the wis2downloader service and will show you the status of the downloads that are currently in progress.

On the top left corner you can see the subscriptions that are currently active.

Keep this dashboard open as you will use it to monitor the download progress in the next exercise.

2.11.4 Reviewing the wis2downloader configuration

The wis2downloader-service started by the wis2box-stack can be configured using the environment variables defined in your wis2box.env file.

The following environment variables are used by the wis2downloader:

- `DOWNLOAD_BROKER_HOST`: The hostname of the MQTT broker to connect to. Defaults to `globalbroker.meteo.fr`
- `DOWNLOAD_BROKER_PORT`: The port of the MQTT broker to connect to. Defaults to 443 (HTTPS for websockets)
- `DOWNLOAD_BROKER_USERNAME`: The username to use to connect to the MQTT broker. Defaults to `everyone`
- `DOWNLOAD_BROKER_PASSWORD`: The password to use to connect to the MQTT broker. Defaults to `everyone`
- `DOWNLOAD_BROKER_TRANSPORT`: `websockets` or `tcp`, the transport-mechanism to use to connect to the MQTT broker. Defaults to `websockets`,
- `DOWNLOAD_RETENTION_PERIOD_HOURS`: The retention period in hours for the downloaded data. Defaults to 24
- `DOWNLOAD_WORKERS`: The number of download workers to use. Defaults to 8. Determines the number of parallel downloads.
- `DOWNLOAD_MIN_FREE_SPACE_GB`: The minimum free space in GB to keep on the volume hosting the downloads. Defaults to 1.

To review the current configuration of the `wis2downloader`, you can use the following command:

```
cat ~/wis2box/wis2box.env | grep DOWNLOAD
```

Review the configuration of the `wis2downloader`

What is the default MQTT broker that the `wis2downloader` connects to?

What is the default retention period for the downloaded data?

Click to reveal answer

The default MQTT broker that the `wis2downloader` connects to is `globalbroker.meteo.fr`.

The default retention period for the downloaded data is 24 hours.

Updating the configuration of the `wis2downloader`

To update the configuration of the `wis2downloader`, you can edit the `wis2box.env` file. To apply the changes you can re-run the start command for the `wis2box-stack`:

```
python3 wis2box-ctl.py start
```

And you will see the `wis2downloader` service restart with the new configuration.

You can keep the default configuration for the purpose of this exercise.

2.11.5 Adding subscriptions to the `wis2downloader`

Inside the **wis2downloader** container, you can use the command line to list, add and delete subscriptions.

To login to the **wis2downloader** container, use the following command:

```
python3 wis2box-ctl.py login wis2downloader
```

Then use the following command to list the subscriptions that are currently active:

```
wis2downloader list-subscriptions
```

This command returns an empty list since no subscriptions are currently active.

For the purpose of this exercise, we will subscribe to the following topic `cache/a/wis2/de-dwd-gts-to-wis2/#`, to subscribe to data published by the DWD-hosted GTS-to-WIS2 gateway and downloading notifications from the Global Cache.

To add this subscription, use the following command:

```
wis2downloader add-subscription --topic cache/a/wis2/de-dwd-gts-to-wis2/#
```

Then exit the **wis2downloader** container by typing `exit`:

```
exit
```

Check the wis2downloader dashboard in Grafana to see the new subscription added. Wait a few minutes and you should see the first downloads starting. Go to the next exercise once you have confirmed that the downloads are starting.

2.11.6 Viewing the downloaded data

The wis2downloader-service in the wis2box-stack downloads the data in the 'downloads' directory in the directory you defined as the WIS2BOX_HOST_DATADIR in your wis2box.env file. To view the contents of the downloads directory, you can use the following command:

```
ls -R ~/wis2box-data/downloads
```

Note that the downloaded data is stored in directories named after the topic the WIS2 Notification was published on.

2.11.7 Removing subscriptions from the wis2downloader

Next, log back in to the wis2downloader container:

```
python3 wis2box-ctl.py login wis2downloader
```

and remove the subscription you made from the wis2downloader, using the following command:

```
wis2downloader remove-subscription --topic cache/a/wis2/de-dwd-gts-to-wis2/#
```

And exit the wis2downloader container by typing `exit`:

```
exit
```

Check the wis2downloader dashboard in Grafana to see the subscription removed. You should see the downloads stopping.

2.11.8 Download and decode data for a tropical cyclone track

In this exercise, you will subscribe to the WIS2 Training Broker which is publishing example data for training purposes. You will setup a subscription to download data for a tropical cyclone track. You will then decode the downloaded data using the "decode-bufr-jupyter" container.

Subscribe to the wis2training-broker and setup a new subscription

This demonstrates how to subscribe to a broker that is not the default broker and will allow you to download some data published from the WIS2 Training Broker.

Edit the wis2box.env file and change the DOWNLOAD_BROKER_HOST to `wis2training-broker.wis2dev.io`, change DOWNLOAD_BROKER_PORT to `1883` and change DOWNLOAD_BROKER_TRANSPORT to `tcp`:

```
# downloader settings
DOWNLOAD_BROKER_HOST=wis2training-broker.wis2dev.io
DOWNLOAD_BROKER_PORT=1883
DOWNLOAD_BROKER_USERNAME=everyone
DOWNLOAD_BROKER_PASSWORD=everyone
# download transport mechanism (tcp or websockets)
DOWNLOAD_BROKER_TRANSPORT=tcp
```

Then run the 'start'-command again to apply the changes:

```
python3 wis2box-ctl.py start
```

Check the logs of the wis2downloader to see if the connection to the new broker was successful:

```
docker logs wis2downloader
```

You should see the following log message:

```
...
INFO - Connecting...
INFO - Host: wis2training-broker.wis2dev.io, port: 1883
INFO - Connected successfully
```

Now we will setup a new subscription to the topic to download cyclone-track data from the WIS2 Training Broker.

Login to the **wis2downloader** container:

```
python3 wis2box-ctl.py login wis2downloader
```

And execute the following command (copy-paste this to avoid typo's):

```
wis2downloader add-subscription --topic origin/a/wis2/int-wis2-  
training/data/core/weather/prediction/forecast/medium-range/probabilistic/trajectory
```

Exit the **wis2downloader** container by typing `exit`.

Wait until you see the downloads starting in the wis2downloader dashboard in Grafana.



Downloading data from the WIS2 Training Broker

The WIS2 Training Broker is a test broker that is used for training purposes and may not publish data all the time.

During the in-person training sessions, the local trainer will ensure the WIS2 Training Broker will publish data for you to download.

If you are doing this exercise outside of a training session, you may not see any data being downloaded.

Check that the data was downloaded by checking the wis2downloader logs again with:

```
docker logs wis2downloader
```

You should see a log message similar to the following:

```
[...] INFO - Message received under topic origin/a/wis2/int-wis2-  
training/data/core/weather/prediction/forecast/medium-range/probabilistic/trajectory  
[...] INFO - Downloaded A_JSXX05ECEP020000_C_ECMP_...
```

Decoding downloaded data

In order to demonstrate how you can decode the downloaded data, we will start a new container using 'decode-bufr-jupyter' image.

This container will be start Jupyter notebook server on your instance which includes the "ecCodes" library that you can use to decode BUFR data.

We will use the example notebooks included in `~/exercise-materials/notebook-examples` to decode the downloaded data for the cyclone tracks.

To start the container, use the following command:

```
docker run -d --name decode-bufr-jupyter \  
-v ~/wis2box-data/downloads:/root/downloads \  
-p 8888:8888 \  
-e JUPYTER_TOKEN=dataismagic! \  
mlimper/decode-bufr-jupyter
```

About the decode-bufr-jupyter container

The `decode-bufr-jupyter` container is a custom container that includes the `ecCodes` library and runs a Jupyter notebook server. The container is based on a image that includes the `ecCodes` library for decoding BUFR data, along with libraries for plotting and data analysis.

The command above starts the container in detached mode, with the name `decode-bufr-jupyter`, the port 8888 is mapped to the host system and the environment variable `JUPYTER_TOKEN` is set to `dataismagic!`.

The command above also mounts the `~/wis2box-data/downloads` directory to `/root/downloads` in the container. This ensures that the downloaded data is available to the Jupyter notebook server.

Once the container is started, you can access the Jupyter notebook server by navigating to `http://YOUR-HOST:8888` in your web browser.

You will see a screen requesting you to enter a "Password or token".

Provide the token `dataismagic!` to login to the Jupyter notebook server.

After you login, you should see the following screen listing the directories in the container:

Files

Running

Select items to perform actions on them.

New

Upload

Refresh

/

☐	Name	Modified	File Size
☐	downloads	21 hours ago	
☐	example-notebooks	4 minutes ago	
☐	sample-data	1 hour ago	

Double click on the `example-notebooks` directory to open it.

You should see the following screen listing the example notebooks, double click on the `tropical_cyclone_track.ipynb` notebook to open it:

 Files

 Running

Select items to perform actions on them.

 New  Upload 

 / example-notebooks /

☐	Name	Modified	File Size
☐	 example_eccodes.ipynb	yesterday	1.2 KB
☐	 tropical_cyclone_track.ipynb	23 hours ago	20.9 KB

You should now be in the Jupyter notebook for decoding the tropical cyclone track data:

 jupyter tropical_cyclone_track Last Checkpoint: 2 minutes ago 

File Edit View Run Kernel Settings Help Not Trusted

          Markdown

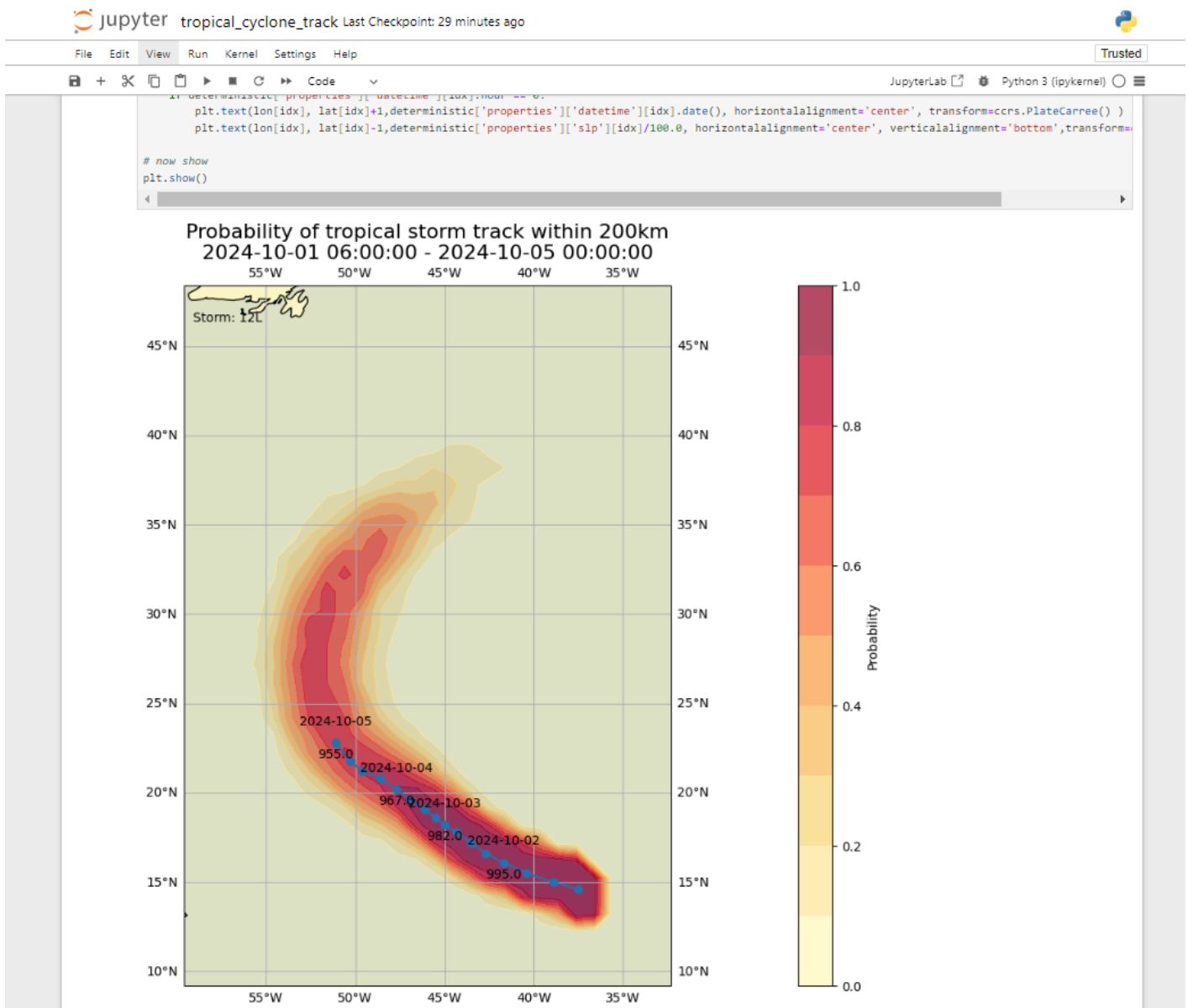
 JupyterLab  Python 3 (ipykernel)

BUFR tropical cyclone example

This notebook provides an example on how to decode BUFR data using the ecCodes library. In this example, we will read tropical cyclone track data from the ECMWF ensemble forecasting system and do a basic analysis to calculate the tropical cyclone strike probability and display the result on a map.

Read the instructions in the notebook and run the cells to decode the downloaded data for the tropical cyclone tracks. Run each cell by clicking on the cell and then clicking the run button in the toolbar or by pressing `Shift+Enter`.

At the end you should see a plot of the strike probability for the tropical cyclone tracks:



? Question

The result displays the predicted probability of tropical storm track within 200 km. How would you update the notebook to display the predicted probability of tropical storm track within 300 km ?

 Click to reveal answer

To update the notebook to display the predicted probability of tropical storm track within a different distance, you can update the `distance_threshold` variable in the code-block that calculates the strike probability.

To display the predicted probability of tropical storm track within 300 km,

```
# set distance threshold (meters)
distance_threshold = 300000 # 300 km in meters
```

Then re-run the cells in the notebook to see the updated plot.

 Decoding BUFR data

The exercise you just did provided one specific example of how you can decode BUFR data using the `ecCodes` library. Different data types may require different decoding steps and you may need to refer to the documentation for the data type you are working with.

For more information please consult the [ecCodes documentation](#).

2.11.9 Conclusion

 Congratulations!

In this practical session, you learned how to:

- use the 'wis2downloader' to subscribe to a WIS2 Broker and download data to your local system
- view the status of the downloads in the Grafana dashboard
- decode some downloaded data using the 'decode-bufr-jupyter' container

2.12 Discovering datasets from the WIS2 Global Discovery Catalogue

Learning outcomes!

By the end of this practical session, you will be able to:

- use pywiscat to discover datasets from the Global Discovery Catalogue (GDC)

2.12.1 Introduction

In this session you will learn how to discover data from the WIS2 Global Discovery Catalogue (GDC).

At the moment, the following GDCs are available:

- Environment and Climate Change Canada, Meteorological Service of Canada: <https://wis2-gdc.weather.gc.ca>
- China Meteorological Administration: <https://gdc.wis.cma.cn>
- Deutscher Wetterdienst: <https://wis2.dwd.de/gdc>

During local training sessions, a local GDC is set up to allow participants to query the GDC for the metadata they published from their wis2box-instances. In this case the trainers will provide the URL to the local GDC.

2.12.2 Preparation

Note

Before starting please login to your student VM.

2.12.3 Installing pywiscat

Use the `pip3` Python package installer to install pywiscat on your VM:

```
pip3 install pywiscat
```

 Note

If you encounter the following error:

```
WARNING: The script pywiscat is installed in '/home/username/.local/bin' which is not on PATH.  
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
```

Then run the following command:

```
export PATH=$PATH:/home/$USER/.local/bin
```

...where `$USER` is your username on your VM.

Verify that the installation was successful:

```
pywiscat --version
```

2.12.4 Finding data with pywiscat

By default, pywiscat connects to Canada's Global Discovery Catalogue. Let's configure pywiscat to query the training GDC by setting the `PYWISCAT_GDC_URL` environment variable:

```
export PYWISCAT_GDC_URL=http://gdc.wis2.training:5002
```

Let's use `pywiscat` to query the GDC setup as part of the training.

```
pywiscat search --help
```

Now search the GDC for all records:

```
pywiscat search
```

 Question

How many records are returned from the search?

 Click to reveal answer

The number of records depends on the GDC you are querying. When using the local training GDC, you should see that the number of records is equal to the number of datasets that have been ingested into the GDC during the other practical sessions.

Let's try querying the GDC with a keyword:

```
pywiscat search -q observations
```

Question

What is the data policy of the results?

Click to reveal answer

All data returned should specify "core" data

Try additional queries with `-q`

Tip

The `-q` flag allows for the following syntax:

- `-q synop` : find all records with the word "synop"
- `-q temp` : find all records with the word "temp"
- `-q "observations AND oman"` : find all records with the words "observations" and "oman"
- `-q "observations NOT oman"` : find all records that contain the word "observations" but not the word "oman"
- `-q "synop OR temp"` : find all records with both "synop" or "temp"
- `-q "obs*"` : fuzzy search

When searching for terms with spaces, enclose in double quotes.

Let's get more details on a specific search result that we are interested in:

```
pywiscat get <id>
```

Tip

Use the `id` value from the previous search.

2.12.5 Conclusion

**Congratulations!**

In this practical session, you learned how to:

- use pywiscat to discover datasets from the WIS2 Global Discovery Catalogue

3 Cheatsheets

3.1 Linux cheatsheet

3.1.1 Overview

The basic concepts of working in a Linux operating system are **files** and **directories** (folders) organized in a tree structure within an **environment**.

Once you login to a Linux system, you are working in a **shell** in which you can work on files and directories, by executing commands which are installed on the system. The Bash shell is a common and popular shell which is typically found on Linux systems.

3.1.2 Bash

Directory Navigation

- Entering an absolute directory:

```
cd /dir1/dir2
```

- Entering a relative directory:

```
cd ./somedir
```

- Move one directory up:

```
cd ..
```

- Move two directories up:

```
cd ../../
```

- Move to your "home" directory:

```
cd -
```

File Management

- Listing files in the current directory:

```
ls
```

- Listing files in the current directory with more detail:

```
ls -l
```

- List the root of the filesystem:

```
ls -l /
```

- Create an empty file:

```
touch foo.txt
```

- Create a file from an `echo` command:

```
echo "hi there" > test-file.txt
```

- View the contents of a file:

```
cat test-file.txt
```

- Copy a file:

```
cp file1 file2
```

- Wildcards: operate on file patterns:

```
ls -l fil* # matches file1 and file2
```

- Concatenate two files into a new file called `newfile`:

```
cat file1 file2 > newfile
```

- Append another file into `newfile`

```
cat file3 >> newfile
```

- Delete a file:

```
rm newfile
```

- Delete all files with the same file extension:

```
rm *.dat
```

- Create a directory

```
mkdir dir1
```

Chaining commands together with pipes

Pipes allow a user to send the output of one command to another using the pipe `|` symbol:

```
echo "hi" | sed 's/hi/bye/'
```

- Filtering command outputs using grep:

```
echo "id,title" > test-file.txt
echo "1,birds" >> test-file.txt
echo "2,fish" >> test-file.txt
echo "3,cats" >> test-file.txt

cat test-file.txt | grep fish
```

- Ignoring case:

```
grep -i FISH test-file.txt
```

- Count matching lines:

```
grep -c fish test-file.txt
```

- Return outputs not containing keyword:

```
grep -v birds test-file.txt
```

- Count the number of lines in `test-file.txt`:

```
wc -l test-file.txt
```

- Display output one screen at a time:

```
more test-file.txt
```

...with controls:

- Scroll down line by line: *enter*
- Go to next page: *space bar*
- Go back one page: *b*
- Display the first 3 lines of the file:

```
head -3 test-file.txt
```

- Display the last 2 lines of the file:

```
tail -2 test-file.txt
```

3.2 Docker cheatsheet

3.2.1 Overview

Docker allows for creating virtual environments in an isolated manner in support of virtualization of computing resources. The basic concept behind Docker is containerization, where software can run as services, interacting with other software containers, for example.

The typical Docker workflow involves creating and building **images**, which are then run as live **containers**.

Docker is used to run the suite of services that make up wis2box using pre-built images.

Image management

- List available images

```
docker image ls
```

- Update an image:

```
docker pull my-image:latest
```

- Removing an image:

```
docker rmi my-image:local
```

Volume Management

- List all created volumes:

```
docker volume ls
```

- Display detailed information on a volume:

```
docker volume inspect my-volume
```

- Remove a volume:

```
docker volume rm my-volume
```

- Remove all unused volumes:

```
docker volume prune
```

Container Management

- Display a list of currently running containers:

```
docker ps
```

- List of all containers:

```
docker ps -a
```

- Enter the interactive terminal of a running container:

**Tip**

use `docker ps` to use the container id in the command below

```
docker exec -it my-container /bin/bash
```

- Remove a container

```
docker rm my-container
```

- Remove a running container:

```
docker rm -f my-container
```


3.3 WIS2 in a box cheatsheet

3.3.1 Overview

wis2box runs as a suite of Docker Compose commands. The `wis2box-ctl.py` command is a utility (written in Python) to run Docker Compose commands easily.

3.3.2 wis2box command essentials

Starting and stopping

- Start wis2box:

```
python3 wis2box-ctl.py start
```

- Stop wis2box:

```
python3 wis2box-ctl.py stop
```

- Verify all wis2box containers are running:

```
python3 wis2box-ctl.py status
```

- Login to a wis2box container (*wis2box-management* by default):

```
python3 wis2box-ctl.py login
```

- Login to a specific wis2box container:

```
python3 wis2box-ctl.py login wis2box-api
```

4 csv2bufr templates

4.1 csv2bufr template for Automated Weather Stations reporting hourly GBON data

The **AWS Template** uses a standardized CSV format to ingest data from Automatic Weather Stations in support of GBON reporting requirements. This mapping template converts CSV data to BUFR sequence 301150, 307096.

The format is intended for use with automatic weather stations reporting a minimum number of parameters, including pressure, air temperature and humidity, wind speed and direction and precipitation on an hourly basis.

4.1.1 CSV columns and description

Column	Units	Data type	Description
wsi_series		Integer	WIGOS identifier series
wsi_issuer		Integer	WIGOS issuer of identifier
wsi_issue_number		Integer	WIGOS issue number
wsi_local		Character	WIGOS local identifier
wmo_block_number		Integer	WMO block number
wmo_station_number		Integer	WMO station number
station_type		Integer	Type of observing station, encoding using code table 0 02 001 (set to 0, automatic)
year		Integer	Year (UTC), the time of observation (based on the actual time the barometer is read)
month		Integer	Month (UTC), the time of observation (based on the actual time the barometer is read)
day		Integer	Day (UTC), the time of observation (based on the actual time the barometer is read)

Column	Units	Data type	Description
hour		Integer	Hour (UTC), the time of observation (based on the actual time the barometer is read)
minute		Integer	Minute (UTC), the time of observation (based on the actual time the barometer is read)
latitude	degrees	Decimal	Latitude of the station (to 5 decimal places)
longitude	degrees	Decimal	Longitude of the station (to 5 decimal places)
station_height_above_msl	meters	Decimal	Height of the station ground above mean sea level (to 1 decimal place)
barometer_height_above_msl	meters	Decimal	Height of the barometer above mean sea level (to 1 decimal place), typically height of station ground plus the height of the sensor above local ground
station_pressure	Pa	Decimal	Pressure observed at the station level to the nearest 10 pascals
msl_pressure	Pa	Decimal	Pressure reduced to mean sea level to the nearest 10 pascals
geopotential_height	gpm	Integer	Geopotential height expressed in geopotential meters (gpm) to 0 decimal places
thermometer_height	meters	Decimal	Height of thermometer or temperature sensor above the local ground to 2 decimal places
air_temperature	Kelvin	Decimal	Instantaneous air temperature to 2 decimal places

Column	Units	Data type	Description
dewpoint_temperature	Kelvin	Decimal	Instantaneous dewpoint temperature to 2 decimal places
relative_humidity	%	Integer	Instantaneous relative humidity to zero decimal places
method_of_ground_state_measurement	code table	Integer	Method of observing the state of the ground, encoded using code table 0 02 176
ground_state	code table	Integer	State of the ground encoded using code table 0 20 062
method_of_snow_depth_measurement	code table	Integer	Method of observing the snow depth encoded using code table 0 02 177
snow_depth	meters	Decimal	Snow depth at time of observation to 2 decimal places
precipitation_intensity	kg m ⁻² h ⁻¹	Decimal	Intensity of precipitation at time of observation to 5 decimal places
anemometer_height	meters	Decimal	Height of the anemometer above local ground to 2 decimal place
time_period_of_wind	minutes	Integer	Defines the time period over which the wind speed and direction have been averaged. Set to -10 to indicate a measurement period over the preceeding 10 minutes.
wind_direction	degrees	Integer	Wind direction (at anemometer height) averaged from the caterisan components over the indicated time period, 0 decimal places
wind_speed	ms ⁻¹	Decimal	Wind speed (at anemometer height) averaged from the

Column	Units	Data type	Description
			cartesian components over the indicated time period, 1 decimal place
maximum_wind_gust_direction_10_minutes	degrees	Integer	Highest 3 second average over the preceeding 10 minutes, 0 decimal places
maximum_wind_gust_speed_10_minutes	ms-1	Decimal	Highest 3 second average over the preceeding 10 minutes, 1 decimal place
maximum_wind_gust_direction_1_hour	degrees	Integer	Highest 3 second average over the preceeding hour, 0 decimal places
maximum_wind_gust_speed_1_hour	ms-1	Decimal	Highest 3 second average over the preceeding hour, 1 decimal place
maximum_wind_gust_direction_3_hours	degrees	Integer	Highest 3 second average over the preceeding 3 hours, 0 decimal places
maximum_wind_gust_speed_3_hours	ms-1	Decimal	Highest 3 second average over the preceeding 3 hours, 1 decimal place
rain_sensor_height	meters	Decimal	Height of the rain gauge above local ground to 2 decimal place
total_precipitation_1_hour	kg m-2	Decimal	Total precipitation over the past hour, 1 decimal place
total_precipitation_3_hours	kg m-2	Decimal	Total precipitation over the past 3 hours, 1 decimal place
total_precipitation_6_hours	kg m-2	Decimal	Total precipitation over the past 6 hours, 1 decimal place

Column	Units	Data type	Description
total_precipitation_12_hours	kg m-2	Decimal	Total precipitation over the past 12 hours, 1 decimal place
total_precipitation_24_hours	kg m-2	Decimal	Total precipitation over the past 24 hours, 1 decimal place

4.1.2 Example

Example CSV file that conforms to the AWS template: [aws-example.csv](#).

4.2 csv2bufr template for daily climate data (DAYCLI)

The **DAYCLI** template provides a standardized CSV format for converting daily climate data to BUFR sequence 307075.

The format is intended for use with Climate Data Management Systems to publish data on WIS2, in support of reporting requirements for daily climate observations.

This templates maps daily observations of:

- Minimum, maximum and average temperature over 24 hours period
- Total accumulated precipitation over 24 hours period
- Total snow depth at time of observation
- Depth of fresh snow over 24 hours period

This template requires additional metadata with respect to the simplified AWS-template: method of calculating average temperature; sensor and station heights; exposure and measurement quality classification.



About the DAYCLI template

Please note that the DAYCLI BUFR sequence will be updated during 2025 to include additional information and revised QC flags. The DAYCLI template included the wis2box will be updated to reflect these changes. WMO will communicate when the wis2box-software is updated to include the new DAYCLI template, to allow users to update their systems accordingly.

4.2.1 CSV columns and description

Column	Units	Data Type	Description
wsi_series		Integer	WIGOS Identifier series, set to 0 for stations
wsi_issuer		Integer	WIGOS Identifier issue, ISO 3 digit (number) country code or 20000 series
wsi_issue_number		Integer	WIGOS Identifier issue number
wsi_local		Character	WIGOS Identifier local identifier, alphanumeric, max 16 characters
wmo_block_number		Integer	WMO block number for station 0 - 99

Column	Units	Data Type	Description
wmo_station_number		Integer	WMO station number 0 - 999
latitude	Degrees	Decimal	Latitude of the station (to 5 decimal places)
longitude	Degrees	Decimal	Longitude of the station (to 5 decimal places)
station_height_above_msl	Meters	Decimal	Height of the station ground above mean sea level (to 1 decimal place)
temperature_siting_classification		Integer	Combined sensor siting and measurement quality classification (temperature). See references for siting classification
precipitation_siting_classification		Integer	Combined sensor siting and measurement quality classification (precipitation). See references for siting classification
averaging_method		Integer	Method used to calculate daily average temperature
year		Integer	Year (UTC) of nominal reporting day
month		Integer	Month (UTC) of nominal reporting day
day		Integer	Day (UTC) of month for nominal reporting day
precipitation_day_offset		Integer	Start of reporting period for precipitation, offset in days relative to nominal reporting day (-1 or 0)
precipitation_hour		Integer	Beginning hour (UTC) over which the precipitation is measured
precipitation_minute		Integer	Beginning minute (UTC) over which the precipitation is measured

Column	Units	Data Type	Description
precipitation_second		Integer	Beginning second (UTC) over which the precipitation is measured
precipitation	kg m-2	Decimal	Total accumulated precipitation over indicated 24 hour period (kg m-3, equivalent to mm)
precipitation_flag		Integer	Quality control flag for total accumulated precipitation
fresh_snow_day_offset		Integer	Start of reporting period for fresh snow depth, offset in days relative to nominal reporting day (-1 or 0)
fresh_snow_hour		Integer	Beginning hour (UTC) over which the fresh snow depth is measured
fresh_snow_minute		Integer	Beginning minute (UTC) over which the fresh snow depth is measured
fresh_snow_second		Integer	Beginning second (UTC) over which the fresh snow depth is measured
fresh_snow_depth	Meters	Decimal	Fresh snow depth over indicated 24 hour period (m, 2 d.p.)
fresh_snow_depth_flag		Integer	Quality control flag for the fresh snow depth
total_snow_day_offset		Integer	Start of reporting period for total snow depth, offset in days relative to nominal reporting day (-1 or 0)
total_snow_hour		Integer	Beginning hour (UTC) over which the total snow depth is measured
total_snow_minute		Integer	Beginning minute (UTC) over which the total snow depth is measured

Column	Units	Data Type	Description
total_snow_second		Integer	Beginning second (UTC) over which the total snow depth is measured
total_snow_depth	Meters	Decimal	Total snow depth over indicated 24 hour period (m, 2 d.p.)
total_snow_depth_flag		Integer	Quality control flag for the total snow depth
thermometer_height	Meters	Decimal	Height of thermometer above local ground level (m)
maximum_temperature_day_offset		Integer	Start of reporting period for maximum temperature, offset in days relative to nominal reporting day (-1 or 0)
maximum_temperature_hour		Integer	Beginning hour (UTC) over which the maximum temperature is measured
maximum_temperature_minute		Integer	Beginning minute (UTC) over which the maximum temperature is measured
maximum_temperature_second		Integer	Beginning second (UTC) over which the maximum temperature is measured
maximum_temperature	Kelvin	Decimal	Maximum temperature over indicated 24 hour period (K, 2 d.p.)
maximum_temperature_flag		Integer	Quality control flag for the maximum temperature
minimum_temperature_day_offset		Integer	Start of reporting period for minimum temperature, offset in days relative to nominal reporting day (-1 or 0)
minimum_temperature_hour		Integer	Beginning hour (UTC) over which the minimum temperature is measured
minimum_temperature_minute		Integer	Beginning minute (UTC) over which the minimum temperature is measured

Column	Units	Data Type	Description
minimum_temperature_second		Integer	Beginning second (UTC) over which the minimum temperature is measured
minimum_temperature	Kelvin	Decimal	Minimum temperature over indicated 24 hour period (K, 2 d.p.)
minimum_temperature_flag		Integer	Quality control flag for the minimum temperature
average_temperature_day_offset		Integer	Start of reporting period for average temperature, offset in days relative to nominal reporting day (-1 or 0)
average_temperature_hour		Integer	Beginning hour (UTC) over which the average temperature is measured
average_temperature_minute		Integer	Beginning minute (UTC) over which the average temperature is measured
average_temperature_second		Integer	Beginning second (UTC) over which the average temperature is measured
average_temperature	Kelvin	Decimal	Average temperature over indicated 24 hour period (K, 2 d.p.)
average_temperature_flag		Integer	Quality control flag for the average temperature

4.2.2 Averaging method

Integer	Description
0	Average of minimum and maximum temperatures
1	Average of 8 tri-hourly observations
2	Average of 24 hourly observations

Integer	Description
3	Weighted average of three observations
4	Weighted average of three observations, as well as the minimum and maximum values
5	AWS complete integration from minute data
6	Average of 4 six-hourly observations

4.2.3 Quality flag

Integer	Description
0	Data checked and declared good
1	Data checked and declared suspect
2	Data checked and declared aggregated
3	Data checked and declared out of instrument range
4	Data checked and declared aggregated and out of instrument range
5	Parameter is not measured at the station
6	Daily value not provided
7	Data unchecked

4.2.4 References for siting classification

[Reference for "temperature_siting_classification".](#)

[Reference for "precipitation_siting_classification".](#)

4.2.5 Example

Example CSV file that conforms to the DAYCLI template: [daycli-example.csv](#).